

MODULE 00 · COURSE OVERVIEW - ALL DOMAINS

AI for Data Engineers

Thirty two prompts for warehouse work, and the reasoning behind every one.

MODULES 11 RUNTIME under 2 hours PROMPTS 32

Around **91%** on the demo database. Close to **zero** on a real warehouse.

What you will actually learn to do

- ◆ Find where documents and code disagree. Before you trust either one.
- ◆ Build a query from a spec without inheriting its ambiguities silently.
- ◆ Debug a wrong number by asking for counts rather than opinions.
- ◆ Prove a rewrite is equivalent instead of being told that it is.
- ◆ Leave behind tests, monitors and a document that is true today.

HOW IT IS TAUGHT

Every prompt appears in full, on screen, and works on any stack.

How the eleven modules run

01

Context

What to hand over, and what is missing from it.

02

Build

Spec to SQL, and reading code nobody documented.

03

Fix

Wrong numbers, slow queries, moving between engines.

04

Ship

Tests, monitors, documentation, and review that works.

05

Beyond SQL

Spark, notebooks and orchestration.

IN PRACTICE

Roughly ten minutes each. They build on each other, so run them in order the first time.

The example we use throughout

WHAT YOU GET

A real project

- A mapping document from go-live
- A spec maintained for two years after
- The stored procedures that actually run
- All three disagreeing with each other

WHAT IS IN IT

Real drift

- A metric defined nowhere in writing
- A typo fixed in one file of three
- A filter that silently does nothing
- None of it planted for teaching

ON THE MATERIAL

Names and details are changed. The failure modes are exactly as found.

Who this is for

- ◆ **You write SQL for a living** against a warehouse somebody else designed.
- ◆ **You have used AI for this** and been burned by output that looked right.
- ◆ **Not a beginner course.** Joins, grain and dimensional modelling are assumed.
- ◆ **Not a tool tutorial.** Nothing depends on which model or vendor you use.

BE HONEST WITH YOURSELF

If you have never had a query return a plausible wrong number, this will not land.

Every prompt hands you something
checkable, and then stops.

Module one is about what you hand over
before you ask for anything.

MODULE 01 · DOMAIN 1 - CONTEXT AND RECONCILIATION

The Context Package

Why the demo worked on their database and fails on yours.

DOMAIN 1 of 5 MODULE 1 of 11 PROMPTS 2

What this course is, and is not

- ◆ **It is about judgement.** Thirty two prompts, and why each is shaped the way it is.
- ◆ **It is warehouse work.** Mapping documents, stored procedures, grain, joins, drift.
- ◆ **It is not a tool tutorial.** Nothing here depends on which model you use.
- ◆ **Everything is copy and paste.** Every prompt appears in full, on screen.

WHY IT IS BUILT THIS WAY

The prompts will age. The reasons they are shaped this way will not.

Accuracy does not decline as databases get realistic. It **collapses**.

The same task, four benchmarks

BENCHMARK	ACCURACY	WHAT CHANGED
Spider	around 91%	Small, clean, tidy schemas
BIRD	around 77%	Larger schemas, messier values
Spider 2.0	roughly 17 to 21%	Real enterprise schemas
BEAVER	close to zero	Real warehouse, real queries

THE POINT

Same task, same models. Only the databases changed.

What actually drives that

- ◆ **Column counts.** Benchmark schemas average tens of columns. Yours run to hundreds.
- ◆ **Join depth.** Benchmark queries need about one join. Under one percent need four.
- ◆ **Naming.** Benchmark columns are called what they are. Yours are called MTART.
- ◆ **None of that is a model problem.** It is a context problem, and you control it.

THE POINT

The gap between the demo and your job is almost entirely the schema you did not hand over.

Two databases

THE DEMO

Tidy

- Around 28 columns
- Column names describe the data
- One join, usually zero
- One meaning per column

YOUR WAREHOUSE

Real

- Hundreds of columns
- MATNR, WERKS, LGORT, BWART
- Six joins across four schemas
- One column meaning three things

THE TIEBREAKER

You cannot make your warehouse tidy. You can make the part you hand over legible.

Assemble the context. Do not describe it.

Three things, in this order

- ◆ **The DDL, verbatim.** Create table statements, pasted. Not a summary.
- ◆ **Five sample rows per table.** Real values, or realistic ones.
- ◆ **The lookup contents.** If a column holds codes, paste the code table.
- ◆ **Then the question.** Last, not first.

WHY DDL AND NOT PROSE

The research calls it schema linking. The model has to connect your words to exact column names.

Two ways to describe the same table

Both are accurate. Only one is usable.

```
01 -- What people paste:
02 "The parts table has a part number, a site, stock levels,
03  and a planning type field."
04
05 -- What actually works:
06 CREATE TABLE pmats (
07   partno  VARCHAR(40), site   VARCHAR(10),
08   stloc   VARCHAR(10), plntyp VARCHAR(6),
09   minqty  NUMERIC(18,6), maxqty NUMERIC(18,6) );
10
11 -- The prose never mentions stloc. It is in every filter.
```

WHAT IT COST

The summary dropped the stock location column. Every subsequent query would ignore it.

Why sample rows matter more than they look

- ◆ **They show what a code looks like.** Is the status 'A' or 'ACTIVE' or 1?
- ◆ **They show the real types.** A date column that is actually a string.
- ◆ **They show null behaviour.** Real nulls, or empty strings pretending to be nulls.
- ◆ **Half of all wrong output** traces to an assumption about values, not structure.

THE POINT

Five rows per table. It takes two minutes and it removes a whole category of error.

What you actually type

The framing above the paste is short. The paste is the work.

Below is the schema I am working with. Use only these tables and columns. Do not assume any column that is not listed.

1. CREATE TABLE statements, verbatim
2. Five sample rows per table
3. Contents of every lookup table referenced

If you need a column that is not here, stop and ask. Do not substitute a column that looks similar.

I will give you the actual task in my next message. For now, tell me what you understand this schema to represent.

THE LINE THAT CATCHES PROBLEMS EARLY

A cheap test. A wrong summary means everything after it would be wrong.

Before you ask for anything, ask what is
missing.

What the gap audit finds

- ◆ **Lookup tables you forgot.** A code column with no code table pasted.
- ◆ **Relationships you assumed.** Two tables that must join, with no obvious key.
- ◆ **Undefined values.** A status column whose codes appear nowhere.
- ◆ **Cheaper now than later.** Two minutes here, or an afternoon of wrong output.

THE POINT

It is asking what it would need to guess at. Guesses are what you are trying to prevent.

What you actually type

Run it after the schema, before the task.

```
Here is the task I am about to give you: <one line summary>
```

```
Do not attempt it. Instead, tell me what is missing from the  
schema I pasted that you would need in order to do it.
```

```
Specifically:
```

- Any column referenced by the task but absent
- Any code or status column with no lookup provided
- Any join you would need where the key is not obvious
- Anything you would have to assume about the data

```
If nothing is missing, say so. Do not invent gaps.
```

THE SHAPE

Give it the task in one line so it knows what to check for, then forbid it from starting.

What it returned on a real schema

Nine tables pasted, one task described

- ◆ **Two missing lookups.** Movement type and part type codes, both filtered on, neither defined.
- ◆ **One ambiguous join.** Two tables sharing three columns. Which are the key?
- ◆ **Four assumptions** it would have made silently, now listed.
- ◆ **Total time to fix.** About six minutes of pasting.

THE RESULT

Every one of those would have become a confident, invisible decision.

Your fact table has 140 columns. The task involves eight of them. You are about to paste the schema.

Pasting all 140 uses a lot of the context window and includes a great deal that is irrelevant.

What do you paste?

- ☐ A All 140. More context is safer
- ☐ B The 8 you need. Everything else is noise
- ☐ C The 8, plus keys, plus anything named similarly to the 8
- ☐ D A summary of all 140, plus full DDL for the 8

◆ PAUSE AND DECIDE

What do you paste?

- A

 All 140
More context is not reliably better. Quality degrades as irrelevant material accumulates.
- B

 Just the 8
Too tight. It cannot join without keys, and it cannot warn you about a column you overlooked.
- C

 The 8, keys, and near-names
Enough to work with, and near-names let it flag a column you may have picked wrongly.
- D

 Summary plus DDL
The summary of 132 columns adds length without precision. That is the worst of both.

THE SIGNAL

Similar names matter. If there is a `total_qty` and a `total_qty_net`, you want to be told.

You paste the schema and ask what it represents. The summary comes back and describes your usage fact table as a snapshot of current stock levels.

It is not. It is a rolling twelve month consumption total.

What does this tell you?

- ☐ A The model is unreliable. Proceed with caution
- ☐ B The table name and columns genuinely suggest a snapshot
- ☐ C The sample rows were not representative
- ☐ D Correct it and move on. It is only a summary

◆ PAUSE AND DECIDE

What does this tell you?

- A

 Model unreliable
It read your schema and reached a reasonable conclusion. That is information about the schema.
- B

 The schema suggests it
The naming genuinely misleads. A new colleague would make the same mistake next month.
- C

 Bad sample rows
Worth checking, and rolling totals and snapshots look identical in five rows.
- D

 Correct it, move on
Fixes this session and leaves the trap in place for everyone else.

THE SIGNAL

The check just told you something true about your warehouse, not about the model.

The gap audit says it needs the lookup table for a status column in order to write the filter correctly.

You know the two codes that matter. Typing them takes ten seconds. Exporting the lookup table takes twenty minutes.

What do you do?

- ☐ A Type the two codes you know. It is what the filter needs
- ☐ B Export the full lookup table
- ☐ C Type the two codes, and note that the list is unverified
- ☐ D Ask the model to infer the codes from the sample rows

◆ PAUSE AND DECIDE

What do you do?

- A

 Type the two codes
Fine for the query and it launders your memory into something that reads as verified.
- B

 Export the full table
Thorough, and twenty minutes is a real cost for a filter you already understand.
- C

 Type them, mark unverified
Keeps the speed and keeps the uncertainty attached to the output.
- D

 Infer from samples
Five rows cannot tell you which codes exist, only which happened to appear.

THE SIGNAL

The pattern recurs all course. Use what you know, and record that you were the source.

The gap audit comes back saying nothing is missing. Your schema paste covers everything needed.

This is the first time you have run it and the task is a six join aggregation across four schemas.

How do you read that?

- ☐ A Good. The context package was complete
- ☐ B Suspicious. Push on the joins specifically
- ☐ C Meaningless. It always says that
- ☐ D Re-run it with the task described in more detail

◆ PAUSE AND DECIDE

How do you read that?

- A

 Package was complete
Possible on a simple task. On six joins across four schemas it is worth thirty seconds of doubt.
- B

 Push on the joins
Ask it to state each join key explicitly. Ambiguity surfaces there before anywhere else.
- C

 It always says that
It does not. The prompt forbids inventing gaps, and it reports them when they exist.
- D

 More task detail
Reasonable and vague. B tests the specific thing most likely to be wrong.

THE SIGNAL

Same shape as a grain declaration with no fan-out risk. A clean answer on a complex task earns a push.

Take this to work tomorrow

- ◆ Paste DDL, never a description. Five sample rows per table.
- ◆ Include lookup contents for every code column you filter on.
- ◆ Ask what the schema represents before you ask for anything else.
- ◆ Run the gap audit. Six minutes of pasting beats an afternoon of plausible output.

THE PAYOFF

None of this is prompting. It is preparation, and it decides everything downstream.

You have the schema. Next: the documents that describe it, and why they are **wrong**.

MODULE 02 · DOMAIN 1 - CONTEXT AND RECONCILIATION

Reconciliation

Finding where the documentation and the deployed code stopped agreeing.

DOMAIN 1 of 5 MODULE 2 of 11 PROMPTS 3

A mapping document is not the specification. It is a **photograph** of one.

Why drift is the default state

This is structural, not a failure of diligence

- ◆ **Documents are written once.** At go live, to get sign off. There is no trigger to revisit them.
- ◆ **Code changes continuously.** Every fix, every enhancement, every source migration.
- ◆ Nobody is slacking. The workflow simply never had a reconciliation step.
- ◆ It never needed one, because the engineer writing the query already knew the document was wrong.

THE REAL CAUSE

Feeding documents to a model removes the person who carried that knowledge.

What each artifact actually knows

THE DOCUMENT KNOWS

Intent

- Why a rule exists at all
- What the business meant by a term
- Which decisions were deliberate
- What was ruled out and why

THE CODE KNOWS

Behaviour

- What actually runs tonight
- Every change since go live
- The literals and codes in force
- Nothing about why any of it is there

THE TIEBREAKER

Neither is authoritative on its own. Reconciliation is how you find out where they diverge.

Three prompts, in this order

01

Diff

Where both sources describe the same thing and say different things.

02

Literals

Where the values inside a matching rule have drifted apart.

03

Orphans

Where only one source mentions something at all.

IN PRACTICE

Run them separately. A combined prompt returns a shorter, worse answer to all three.

The document to code diff.

The output schema you demand

COLUMN	WHY IT IS THERE
Attribute	Anchors every row to something checkable
Document says	Verbatim. Not a paraphrase
Code does	Verbatim. Not a paraphrase
Type	SOURCE, JOIN, FILTER, TRANSFORM, NAME or ABSENT
Confidence	HIGH if unambiguous, LOW if it depends on missing information

THE POINT

Specify the shape or you get prose, and prose about a mapping document is always reassuring.

What you actually type

First half. The framing and the output shape.

```
You are reconciling a data warehouse mapping document against  
the stored procedure that was actually deployed. The document  
was written at go live and has not been maintained. The code  
has changed since. Assume both may be wrong.
```

```
SCOPE: <target table name> only. Ignore all other tables.
```

```
Produce a table with exactly these columns:
```

```
| Attribute | Document says | Code does | Type | Confidence |
```

THE TWO LOAD-BEARING LINES

Naming the scope and the columns is what separates this from a request for a summary.

The rules that make it work

Second half. Paste both halves as one message.

Rules:

- One row per attribute where the two disagree. No matches.
- Quote both sides verbatim, not your paraphrase.
- Type is one of: SOURCE, JOIN, FILTER, TRANSFORM, NAME, ABSENT.
- Confidence is HIGH if unambiguous, LOW if it depends on something not in either file.

After the table, add a section headed CANNOT VERIFY:
anything the document references that is not present in the
material I gave you. Be specific about what is missing.

Do not recommend fixes. Do not summarise.

USE IT AS WRITTEN

Every line exists because dropping it produced a worse answer.

Four design decisions in that prompt

- ◆ **Verbatim, not paraphrase.** A paraphrase of two different things converges on what they share.
- ◆ **One target table per run.** Seven sheets at once gives the same output spread thinner.
- ◆ **Do not list matches.** Otherwise four findings hide among forty non findings.
- ◆ **No recommendations.** A model asked to fix things shades its findings toward what it can fix.

THE POINT

Every one of these is a line in the prompt. None of them is optional.

What it found on the parts usage fact

Five disagreements on one target table

- ◆ **Source schema.** Document says one view layer. Code reads a different one.
- ◆ **Consumption formula.** Document says a simple sum. Code is issues minus returns.
- ◆ **Part hash key.** Document omits the source system key and the upper case wrapper.
- ◆ **Part type filter.** Document and code disagree by a single character.

THE RESULT

Ten minutes of runtime, on a document that had drifted for two years.

The filter that only half applies

Backlog load. Read the last two lines together.

```
01 WHERE NVL(p mats.stloc, '') = '1000'
02       OR (NVL(p mats.stloc, '') = '2000'
03           AND NVL(p mat.ptyp, '') IN ('ZRPR', 'ZCON'))
04       AND NVL(p mats.chgtype, '') <> 'D'
05
06 -- AND binds tighter than OR, so this parses as:
07 --   stloc='1000' OR ( 2000-branch AND not-deleted )
08 -- The delete filter never applies to location 1000.
```

HOW YOU KNOW

Both sibling procedures parenthesise this correctly. That is what establishes intent.

The section headed **cannot verify** is the reason this prompt exists.

What the diff could not resolve

- ◆ It flagged that the consumption formula differs. Correctly.
- ◆ It could not tell you which version is right.
- ◆ The code filters on sixteen movement type codes that appear in neither file.
- ◆ A good run says exactly that, and names what is missing.

THE POINT

It did not solve the problem. It turned an invisible problem into a specific question.

Three failures worth causing on purpose

Break the prompt deliberately so you recognise the symptom

- ◆ **Drop the output schema.** You get a paragraph of reassurance.
- ◆ **Drop the word verbatim.** Two different formulas get paraphrased into the same sentence.
- ◆ **Ask for fixes too.** The cannot verify section vanishes, because nothing in it is actionable.

WATCH FOR

The third one is the dangerous failure. You lose the finding that mattered most.

The literal sweep.

What you actually type

Shorter than it looks. The third paragraph is the one people leave out.

List every hard coded literal used in a filter, join, or CASE condition across all procedures in this file. Include quoted strings, numeric codes and date constants. Exclude literals used only as output values or default assignments.

Group the output BY VALUE, not by procedure. For each value:

Value | Column compared against | Procedures | Context

Then add NEAR MISSES: any pair of values above that differ by one character, a transposition or a case change, AND are compared against the same column.

Then add MULTI-LIST VALUES: any value appearing in more than one list compared against the same column in one statement.

Do not evaluate correctness. Report only.

DO NOT SKIP THE THIRD BLOCK

Drop the last section and movement type 262 survives every pass you will ever run.

The one design decision that matters

GROUPED BY PROCEDURE

The obvious way

- One section per stored procedure
- Matches how you read the file
- Two similar codes end up pages apart
- Nothing connects them

GROUPED BY VALUE

The way that works

- One row per distinct literal
- Sorted, so near matches sit together
- The problem is visible before you read
- Costs nothing extra to ask for

THE TIEBREAKER

Group by value and the defect becomes a visual pattern rather than a reading exercise.

What the sweep returned

VALUE	COMPARED AGAINST	APPEARS IN
ZRPS	part type	Parts usage load
ZRPR	part type	Part site load, backlog load
1000	stock location	All three loads
2000	stock location	All three loads
262	movement type	Issue list AND return list

THE FINDING

One code was corrected in a single procedure eighteen months ago. Two others were missed.

The value that appears in both lists

- ◆ Movement type 262 is in the issue list and in the return list.
- ◆ It gets added, then subtracted. It contributes nothing.
- ◆ Not a near miss. Not a document disagreement. Invisible to both other prompts.
- ◆ One extra section in the prompt catches it. Ask for values appearing in more than one list.

WHY IT HIDES

The query runs. The number is wrong. Nothing about the output would ever tell you.

One value, two lists

Consumption is issues minus returns. Watch movement type 262.

```
01 SUM(CASE WHEN mvttyp IN
02      ('261','262','291','301','311','321','601','641','961')
03      THEN qty ELSE 0 END)          -- issues
04 -
05 SUM(CASE WHEN mvttyp IN
06      ('262','292','302','312','322','602','642')
07      THEN qty ELSE 0 END)          -- returns
08
09 -- 262 is added, then subtracted. It contributes nothing.
```

THE REAL PROBLEM

Sixteen codes, and not one of them appears in the mapping document or the spec.

Where the literal sweep breaks

- ◆ **Dynamic SQL is invisible.** Anything assembled at runtime will not appear.
- ◆ The model will not tell you it looked and found nothing.
- ◆ If the codebase uses execute statements, say so and expect an incomplete sweep.
- ◆ Incomplete and known is fine. Incomplete and assumed complete is not.

THE BLIND SPOT

Know the blind spot, because the output looks identical either way.

The orphan sweep. Run it in **both** directions.

What you actually type

The authority line in paragraph two prevents a silent guess.

```
Compare the columns in the mapping document against the columns  
actually written by the stored procedure, for <target table>.
```

```
Treat the procedure's INSERT column list as the authority for  
what the code writes. Treat the document's Target Attribute  
column as the authority for what was specified.
```

```
Produce three lists:
```

```
  IN CODE, NOT IN DOCUMENT - and what populates each.
```

```
  IN DOCUMENT, NOT IN CODE.
```

```
  WRITTEN BUT NEVER POPULATED - literal NULL or constant.
```

```
Match on column name only. Do not infer that two differently  
named columns are the same thing. If you suspect a rename,  
list it in both places and say so.
```

WHY THE LAST LINE EXISTS

A guessed rename stated as fact closes a question that should stay open.

Three lists, not two

- ◆ **In document, not in code.** Everyone checks this direction.
- ◆ **In code, not in document.** Almost nobody checks this one.
- ◆ **Written but never populated.** Present in the insert, always null or constant.
- ◆ **Do not infer renames.** A guessed rename closes an investigation that should stay open.

THE POINT

Name the insert column list as the authority, or the model picks between three conflicting lists silently.

What the orphan sweep found

- ◆ **Six measures in code, in no document.** Issue and return totals for each time window.
- ◆ Those six are the intermediate values that reveal how consumption is really calculated.
- ◆ **One column written, never populated.** A slow mover flag, listed as deferred in the spec.
- ◆ The second finding is not a defect. It is still worth knowing.

THE COST

Somebody added those six columns to validate the net figure. That decision is recorded nowhere.

Which side is right?

PARTS USAGE FACT

The document is wrong

- Document builds a key with a plus sign
- Code concatenates properly
- Every other sheet agrees with the code
- Document is the outlier

PART SITE DIMENSION

The code is wrong

- Document concatenates properly
- Code builds a value with a plus sign
- Same mistake, opposite side
- Code is the outlier

THE TIEBREAKER

Reconciliation tells you where they disagree. It does not tell you who is correct.

Which of these is the outlier?

Top pair from the document. Bottom pair from the code.

```
01 -- Mapping document, parts usage fact:
02 f_fnv_hash(source_system_part_number + source_system_site_code)
03 -- Deployed code, parts usage fact:
04 f_fnv_hash(v_SourceSystemKey || UPPER(partno) || UPPER(site))
05
06 -- Mapping document, part site dimension:
07 UPPER(TRIM(stloc) || '|' || TRIM(plntyp))
08 -- Deployed code, part site dimension:
09 UPPER(TRIM(pmats.stloc) + '|' + TRIM(pmats.plntyp))
```

THE TIEBREAKER

In Redshift the plus sign is not string concatenation. Both uses of it are wrong.

A planner reports that consumption for one site looks about a third too low. The pipeline has not failed in months and no alerts have fired.

You have the mapping document, the deployed stored procedures, and no access to the source system. The original developer left last year.

What do you do first?

- ☐ A Ask a model to rewrite the consumption query from the mapping document
- ☐ B Run the document to code diff on the consumption fact table
- ☐ C Query the source tables directly and compare row counts
- ☐ D Ask a model to review the stored procedure for bugs

◆ PAUSE AND DECIDE

What do you do first?

- A

 Rewrite from the mapping document
The document is the thing you have not verified. Rewriting from it reproduces whatever is stale in it.
- B

 Run the document to code diff
Cheapest step that tells you whether the document you are about to trust is still true.
- C

 Compare source row counts
Reasonable, but you said you have no source access. Also slower than reading two files you already hold.
- D

 Ask for a bug review
A generic review finds generic problems. It has no way to know what the code was supposed to do.

THE SIGNAL

Nothing has failed and no alert fired. That is the signal that the defect is in a definition, not in the plumbing.

The diff returns a clean table of six disagreements, and a cannot verify section saying the movement type codes used by the calculation appear in neither file.

Your deadline is tomorrow. The Materials Management team who own the code register are two time zones away.

How should you proceed?

- ☐ A Ask the model to infer the correct movement types from the code
- ☐ B Treat the deployed code as authoritative and document what it does
- ☐ C Send the specific question to Materials Management and proceed on the rest
- ☐ D Rerun the diff with a longer, more detailed prompt

◆ PAUSE AND DECIDE

How should you proceed?

- A

 Infer the codes from the code
The code is the only evidence, so this just restates it with false confidence attached.
- B

 Treat the code as authoritative
Tempting and half right. But we saw one code corrected in only one of three procedures, so the code disagrees with itself.
- C

 Send the question, proceed on the rest
The cannot verify section produced one specific, answerable question. Asking it is the work.
- D

 Rerun with a better prompt
No prompt recovers information that is not in the context. This is the lesson of the module.

THE SIGNAL

The prompt already did its job. It told you exactly which question to ask and who to ask.

A colleague argues that reconciliation is wasted effort. The documents are always stale, so the team should stop maintaining them and read the code when they need the truth.

They point out that the code is version controlled, always current, and cannot lie about what it does.

What is the strongest argument against this?

- ☐ A Auditors require the documentation to exist
- ☐ B Reading code is slower than reading a document
- ☐ C The code cannot tell you which behaviours are deliberate
- ☐ D Version control history explains why changes were made

◆ PAUSE AND DECIDE

What is the strongest argument against this?

- A

 Auditors require it
True in many places and completely unpersuasive. It argues for a document nobody reads.
- B

 Reading code is slower
Often true, but speed is not the issue. A slow correct answer beats a fast wrong one.
- C

 Code cannot tell you what is deliberate
The plus sign defect and the intended concatenation look identical in code. Only intent separates them.
- D

 History explains the why
Commit messages rarely carry business reasoning, and a squashed migration erases what little there was.

THE SIGNAL

Every defect in this module looks exactly like a deliberate design decision until something outside the code says otherwise.

You run the orphan sweep and it returns four columns present in the code and absent from the document. Three are clearly internal audit fields. The fourth is a business measure.

You have limited time and a model that will happily produce a plausible definition for any of them.

What is the right next step for the fourth column?

- ☐ A Ask the model to write a definition from how the code populates it
- ☐ B Add it to the mapping document as undocumented, and raise it
- ☐ C Leave it out, since nothing downstream is asking for it
- ☐ D Ask the model whether the column is important

◆ PAUSE AND DECIDE

What is the right next step for the fourth column?

- A

 Generate a definition from the code
Produces a mechanical description of the calculation. It cannot recover why anyone wanted the measure.
- B

 Record it as undocumented and raise it
Preserves the finding, marks it as unresolved, and routes it to someone who can answer.
- C

 Leave it out
Nothing asking for it today is not evidence. Somebody built it deliberately and stopped being asked.
- D

 Ask if it is important
The model has no basis for that judgement and will answer anyway. This is the worst option here.

THE SIGNAL

A generated definition looks exactly like a real one, and it silently closes the question.

Take this to work tomorrow

- ◆ Pick one table you are about to change. Run the diff on it before you touch anything.
- ◆ Run the literal sweep across your whole procedure file. It costs one prompt.
- ◆ Whatever lands in cannot verify becomes a message to a named person, not a to do item.
- ◆ Do not fix anything in the same session. Findings and fixes are separate passes.

THE PAYOFF

Three prompts, ten minutes, on documents nobody has opened in two years.

Reconciliation produces a list of **questions**,
not answers.

MODULE 03 · DOMAIN 2 - GENERATING AND READING SQL

Spec to SQL

Turning a data model, a mapping document and a functional spec into a query that is right.

DOMAIN 2 of 5 MODULE 3 of 11 PROMPTS 3

What the last module left you with

- ◆ A list of places the document and the code disagree.
- ◆ A short list of things nobody could verify from the files.
- ◆ Answers to some of those, from the people who owned them.
- ◆ You have not written a line of SQL yet. That was deliberate.

BEFORE YOU START

If you skipped reconciliation, everything in this module builds on documents you have not checked.

A query that runs, returns plausible numbers, and is **silently** wrong.

Two ways to do this

THE DEFAULT

One big ask

- Paste all three documents
- Ask for the query
- Get two hundred lines back
- Start debugging what you cannot read

WHAT WE BUILD

Four checked stages

- Declare the grain before any SQL
- Surface what cannot be implemented
- Build joins, verify counts, then add
- Every stage verified before the next

THE TIEBREAKER

Both take about the same wall clock time. Only one of them tells you where it went wrong.

Declare the grain. Before any SQL exists.

Why grain comes first

- ◆ **Grain is what one row means.** One work order. One part at one site. One movement line.
- ◆ Get it wrong and every measure inflates. Sums, counts, averages, all of them.
- ◆ The query still runs. The numbers still look reasonable.
- ◆ Catching it here costs one sentence. Catching it after a stakeholder queries a dashboard costs a great deal more.

THE POINT

Wrong grain is the most expensive mistake in this workflow and the cheapest one to prevent.

Three tables, one multiplication

Labour hours per work order. Looks correct. Is not.

```
01 SELECT wauf.wonum,  
02        SUM(wopr.actlhrs) AS labour_hours  
03 FROM   wauf  
04 JOIN   wopr ON wopr.wonum = wauf.wonum   -- many ops per W0  
05 JOIN   wres ON wres.wonum = wauf.wonum   -- many reservations  
06 GROUP BY wauf.wonum  
07  
08 -- wres was joined for a filter, not for a measure.  
09 -- Every operation row is now repeated once per reservation.  
10 -- A work order with 4 ops and 5 parts reports 20 rows of hours.
```

WHY IT SURVIVES REVIEW

Nothing here is a syntax error. Nothing fails. The hours are five times too high.

What you actually type

No SQL yet. That instruction is the whole prompt.

```
Using the mapping document and data model below, do NOT write
any SQL yet. Answer these four questions first.
```

1. What does exactly one row of the output represent?
State it as a sentence beginning 'One row per ...'
2. List the join path you intend to use, in order. For each
join, state whether it can return more than one row per
row of the driving table.
3. For any join that can fan out, say how you will prevent
it inflating the measures.
4. Which columns form the unique key of the output?

```
Answer in under 200 words. Do not include any SQL.
```

THE LOAD-BEARING QUESTION

Question two forces the fan-out into the open, before it can hide in a join.

Reading the answer

ACCEPT THIS

Specific and hedged

- Names the grain in one sentence
- Flags reservations as one to many
- Proposes pre-aggregation, and says why
- Names the unique key columns

SEND IT BACK

Confident and vague

- 'One row per work order with its details'
- Lists joins with no cardinality
- Says joins are standard, no fan-out
- Skips the unique key entirely

THE TIEBREAKER

A grain declaration that flags no risk on a four table join has not been done.

Make it declare what it **could not** implement.

Every spec contains something unbuildable

- ◆ A rule that names a concept but never gives you the codes.
- ◆ A calculation whose wording could mean two different things.
- ◆ A grain the source data cannot actually support.
- ◆ The default behaviour is to pick one reading and say nothing.

THE POINT

Silence is the problem. A model that guessed and told you has done its job.

What you actually type

Run this before you ask for code, not after.

```
Read the specification below against the data model provided.  
Do not write SQL.
```

```
List every rule in the spec that you could NOT implement  
exactly as written from the columns available. For each one:
```

- Quote the rule verbatim
- State precisely what is missing or ambiguous
- Give the readings you could take, and which you would pick
- State what evidence would settle it

```
Then list any rule you CAN implement but only by assuming  
something the spec does not state. Name the assumption.
```

```
If you find none, say so explicitly. Do not invent items.
```

WHY THE LAST LINE EXISTS

The last line matters. Without it a model finds problems to fill the shape you asked for.

What it returned on our sample spec

Four items, from a spec that reads as complete

- ◆ **Movement type codes.** Spec names a register held by another team. Not provided.
- ◆ **Excluded part types.** Obsolete parts and kit headers named as concepts, not codes.
- ◆ **Shortfall grain.** Compared at stock location, but the maximum is only held at site.
- ◆ **Currency.** Listed as an open item. Values are in transaction currency.
- ◆ The last two are in the spec. The model found them by reading it properly.

THE RESULT

Two things you cannot build, and two you can build only if you accept a stated compromise.

Build it in stages. Verify between each one.

Four stages, four checks

01

Joins

Skeleton only. Count rows and compare to the driving table.

02

Filters

Add the WHERE clause. Count again. Explain the drop.

03

Transforms

Add derived columns. Spot check twenty rows by hand.

04

Aggregate

Group and sum. Reconcile one part against the source.

IN PRACTICE

Each stage is a separate prompt, and each one ends with a number you look at.

What you actually type, at stage one

Repeat the shape for each stage, changing only what is added.

We are building this query in four stages. This is stage one.

Write the JOIN skeleton only. No WHERE clause, no derived columns, no aggregation. Select only the join keys.

The declared grain is: <paste the grain sentence from prompt 6>

Also give me a COUNT query I can run against this skeleton, and tell me what row count you expect relative to the driving table. If the counts differ, say which join causes it.

Do not proceed to stage two. Wait for the counts.

THE LINE THAT CONNECTS THE PROMPTS

Pasting the grain sentence back in is what keeps stage one honest about stage zero.

What you are checking between stages

- ◆ **After joins.** Row count against the driving table. Any increase is a fan-out.
- ◆ **After filters.** The drop, and whether you can account for it.
- ◆ **After transforms.** Twenty rows, read by eye, against the source.
- ◆ **After aggregation.** One part, one site, reconciled by hand.

THE POINT

Four numbers. If you cannot explain one of them, stop and find out why before continuing.

When it goes wrong mid-build

WHAT MOST PEOPLE DO

Keep iterating

- Paste the error back in
- Get a revision
- Paste the next error back in
- The context fills with dead ends

WHAT WORKS

Restart the stage

- Take what you learned
- Add it to the original prompt
- Start that stage clean
- Earlier stages stay verified

THE TIEBREAKER

Multi-turn debugging degrades. Staged building means restarting costs you one stage, not the query.

You ask for a grain declaration on a five table join. The answer comes back in ninety words, names the grain clearly, lists all four joins, and reports no fan-out risk on any of them.

The query has not been written yet. You have twenty seconds to decide whether to proceed.

What do you do?

- ☐ A Proceed to stage one. The declaration is clear and complete
- ☐ B Ask it to justify the no fan-out claim for each join individually
- ☐ C Ignore it and check the cardinality yourself in the data model
- ☐ D Ask for the query and check the row counts afterwards

◆ PAUSE AND DECIDE

What do you do?

- A

 Proceed, it is complete
Clear is not the same as correct. Four joins with zero risk is the answer to be suspicious of.
- B

 Ask it to justify each join
Cheapest challenge available. A model asked to defend a claim per join often revises it.
- C

 Check cardinality yourself
Correct, and you should. But it is the slower move and B often surfaces the problem in one prompt.
- D

 Get the query, check counts after
This abandons the technique. Row counts after the fact tell you something is wrong, not where.

THE SIGNAL

A clean answer on a complex join is a signal to push, not a signal to proceed.

The unimplementable rules prompt returns one item. A consumption rule depends on movement type codes that live in a register owned by another team.

That team will take three days to respond. Your deadline is Thursday. The deployed legacy procedure contains a list of sixteen codes.

What is the right move?

- ☐ A Copy the sixteen codes from the legacy procedure and proceed
- ☐ B Copy them, proceed, and mark the query as unverified pending the register
- ☐ C Stop work on that measure until the register arrives
- ☐ D Ask the model to infer the correct codes from the movement data

◆ PAUSE AND DECIDE

What is the right move?

- A

 Copy and proceed
Reasonable engineering, but it launders an unverified list into a verified looking query.
- B

 Copy, proceed, mark unverified
Keeps the deadline and keeps the uncertainty visible until the register settles it.
- C

 Stop until the register arrives
Safest and usually wrong. Three days of blocked work to avoid a risk you can carry openly.
- D

 Infer from the data
The data shows which codes exist, never which ones the business counts as consumption.

THE SIGNAL

The legacy list is evidence, not authority. Using it is fine. Forgetting that is not.

At stage one the join skeleton returns 4.1 million rows. The driving table has 890 thousand. The model predicted the counts would match.

You have the four table join in front of you and the data model beside it.

What is the most useful next prompt?

- ☐ A Tell it the counts differ and ask it to fix the query
- ☐ B Ask which join causes the increase, and to prove it with a count per join
- ☐ C Add DISTINCT and move to stage two
- ☐ D Restart stage one with the actual counts included in the prompt

◆ PAUSE AND DECIDE

What is the most useful next prompt?

- A

 Say it differs, ask for a fix
Starts the iteration loop we want to avoid. You get a revision with no diagnosis attached.
- B

 Ask which join, prove with counts
Turns a symptom into a located cause, and the counts are checkable rather than asserted.
- C

 Add DISTINCT, move on
The most damaging option here. It hides the fan-out and the measures still inflate later.
- D

 Restart with the counts
Sound instinct, wrong moment. Restart when iteration has degraded, not on the first surprise.

THE SIGNAL

Stage one exists so that a count mismatch has only four joins to hide in.

Stage four is done. Aggregated totals reconcile against the source for the part you checked by hand. The query is 180 lines and you did not write any of it.

A colleague asks whether it is ready to hand to the reporting team.

What is still missing?

- ☐ A Nothing. Four stages verified and a hand reconciliation is enough
- ☐ B A second reconciliation on a part with different characteristics
- ☐ C A performance review before it runs at full volume
- ☐ D A record of the assumptions carried in from prompt seven

◆ PAUSE AND DECIDE

What is still missing?

- A

 Nothing is missing
One reconciled part proves the query works for that part. It proves very little else.
- B

 A second reconciliation
You should, and pick a part with returns and multiple locations. Still not the biggest gap.
- C

 A performance review
Fair, and it comes later. A slow correct query is a smaller problem than a fast undocumented one.
- D

 A record of the assumptions
Two unverified inputs are baked into this query and nothing on the query says so.

THE SIGNAL

You just spent a module preventing silent decisions. Do not ship one at the last step.

Take this to work tomorrow

- ◆ Next query you are asked for, write the grain sentence yourself before you prompt.
- ◆ Run the unimplementable rules prompt on the spec. It takes two minutes.
- ◆ Build in four stages. Look at the count after each one.
- ◆ Whatever came back from prompt seven goes in a comment at the top of the query.

THE PAYOFF

The last one is thirty seconds of typing and it is the only part that outlives you.

Three prompts, and not one of them asked for the whole query.

MODULE 04 · DOMAIN 2 - GENERATING AND READING SQL

Inherited Code

Eight hundred lines of stored procedure, no documentation, and the author left last year.

DOMAIN 2 of 5 MODULE 4 of 11 PROMPTS 3

What you are actually facing

- ◆ **It works.** It has run every night for years. That is evidence, not documentation.
- ◆ **It is layered.** Temp tables, unions, a sequence of updates after the insert.
- ◆ **Every oddity looks deliberate.** You cannot tell a workaround from a design choice.
- ◆ **You have to change it anyway.** Usually by Friday.

THE REAL RISK

The risk is not that you fail to understand it. The risk is that you understand it wrongly and confidently.

Two ways in

THE DEFAULT

Read it yourself

- Start at line one
- Lose the thread around line 200
- Build a partial mental model
- Change the part you understood

WHAT WE BUILD

Map, then narrate

- Get the shape before the detail
- Narrate one target at a time
- Find what the code cannot tell you
- Then decide what to change

THE TIEBREAKER

Reading it yourself is not wrong. It is just the slowest possible order to do things in.

Map the structure before you ask what anything **means**.

Why shape comes before meaning

- ◆ A long procedure is a sequence of steps, not a query. Each step produces something.
- ◆ Temp tables, unions and post-insert updates hide the real data flow.
- ◆ The map is short enough to check by eye, so you can trust what comes next.
- ◆ Ask what it means first and you get a confident summary you have no way to verify.

THE POINT

You are buying something checkable before you start relying on something that is not.

What you actually type

Deliberately shallow. Do not let it explain anything yet.

```
Map the structure of the stored procedure below. Do not
explain business meaning or summarise what it is for.
```

```
List every step in execution order. For each step give:
```

- What it does, what it writes to, what it reads from

```
Then list every UNION branch separately, and for each branch
state which source table drives it and which filter selects it.
```

```
Then list any column whose value differs between branches.
```

```
Numbered list. No prose. Under 400 words.
```

THE BLOCK THAT EARNS ITS PLACE

The third block finds polymorphic columns, and that is where the bugs live.

One column, three meanings

Work order dimension. Three union branches, abbreviated.

```
01 -- Branch 1 WHERE wotyp IN ('C1','C2','C3')    corrective
02     NVL(UPPER(wauf.assetid),'') AS wo_ref_id
03 -- Branch 2 WHERE wotyp IN ('P1','P2')          preventive
04     NVL(UPPER(wauf.assetid),'') AS wo_ref_id
05 -- Branch 3 WHERE wotyp IN ('J1','J2')          project
06     NVL(UPPER(wauf.contrid),'') AS wo_ref_id
07
08 -- Downstream, the bridge joins on it with no type filter:
09 INNER JOIN dim_asset_fs ast ON ast.asset_number = wo.wo_ref_id
10 -- A contract id that matches an asset number creates rows.
```

WHAT THE MAP BOUGHT YOU

Two branches put an asset in that column. The third puts a contract. Nothing downstream knows.

Where do the keys come from?

The structure map surfaced all three of these together.

```
01 max_corrective_key  bigint := 0;
02 max_preventive_key  bigint := 40000000;
03 max_project_key     bigint := 70000000;
04 -- SELECT NVL(MAX(work_order_key), ...) INTO ... <- commented out
05
06 TRUNCATE TABLE dim_work_order_fs;
07 ... max_corrective_key + ROW_NUMBER() OVER (order by 1)
08
09 -- order by 1 is a constant. It does not determine an order.
10 -- So every reload assigns different keys to the same rows.
```

THE CONDITION

Safe only if nothing outside this procedure ever stored one of these keys.

Why that matters, and when it does not

- ◆ **Fine if the keys never leave.** Rebuilt each night, joined within the same load.
- ◆ **Broken if anything stored one.** A saved report, an extract, a materialised view.
- ◆ **Also a collision risk.** Exceed forty million corrective orders and you overrun the next band.
- ◆ **One query settles it.** Grep for the key column outside this schema.

THE CHECK

You cannot tell from the code whether this is a defect. You can find out in ten minutes.

Narrate the transformations. One target table at a time.

The trap in narration

- ◆ **Restating the SQL is not narration.** 'Coalesces the field to empty string' tells you nothing.
- ◆ You want the effect on the data, not a reading of the syntax.
- ◆ Ask what values arrive, and what values leave.
- ◆ Ask which rows are dropped, and by which predicate.

THE TEST

If the narration could be produced without seeing the data model, it is a translation, not an explanation.

What you actually type

Scope to one target table. Repeat for each.

For target table <name> only, explain what the procedure does to the data. Work column by column, in INSERT list order.

For each column state:

- Which source column or columns feed it
- What happens to the value on the way through
- What it holds when the source is null or missing

Then separately, list every predicate that removes rows, and say what kind of row each one removes.

Do not restate the SQL in English. If a line's only effect is syntactic, say 'no data effect' and move on.

THE HALF PEOPLE SKIP

The row-removal list finds filters that quietly changed scope years ago.

What narration cannot tell you

- ◆ Whether an inner join was chosen deliberately or pasted from elsewhere.
- ◆ Whether a hard-coded code list is current.
- ◆ Why a filter excludes one storage location and not another.
- ◆ It can tell you all three exist. That is the useful part.

THE BOUNDARY

Same boundary as module two. Behaviour is recoverable from code. Intent is not.

Dead code is a drift signal

Three things in this procedure that never execute

- ◆ **Commented incremental filters.** A timestamp range that no longer applies.
- ◆ **A commented soft-delete block.** So the deleted flag is always the same value.
- ◆ **Cleanup after the exception handler.** The handler re-raises, so it never runs.
- ◆ **Each one marks a decision** somebody made and did not finish or record.

WHAT IT COSTS

Downstream filters on that deleted flag are no-ops. Nobody knows, because they still read correctly.

Blast radius. And an honest warning about it.

The caveat first

- ◆ **A model cannot see your warehouse.** It knows only what you paste in.
- ◆ Asked what depends on a table, it will produce a plausible list from names.
- ◆ That list is a hypothesis. It is not lineage and it never was.
- ◆ Feed it real evidence and the same prompt becomes genuinely useful.

READ THIS FIRST

This is the prompt in the library most likely to give you a confident answer with nothing behind it.

What you actually type

Note the first line. It changes what comes back.

```
I am pasting evidence below, not asking you to recall anything.  
Use only what is in this message.
```

1. The procedure that loads <table>
2. Grep results for <table> across our repository
3. The list of views and reports we know read this schema

```
Given only that, list what would be affected if I changed  
the grain of <table>. For each item state:  
- Which piece of evidence tells you it depends on this table  
- What specifically would break, not just that it would
```

```
Then list what you would need to see to be more confident.
```

```
Do not include anything you cannot point to in the evidence.
```

THE RULE THAT FIXES IT

Every item must cite its evidence. That one rule removes the guessing.

If you have no lineage tooling

Most teams do not. This still works

- ◆ **Grep the repository** for the table name. Crude and surprisingly complete.
- ◆ **Query the system tables** for view definitions referencing it.
- ◆ **Check query history** for who has selected from it in ninety days.
- ◆ Paste all three in. Ten minutes of gathering beats any amount of prompting.

IN PRACTICE

The prompt is only as good as the evidence. Gathering the evidence is the actual work.

The structure map reports that the procedure truncates its target, inserts, and then runs three separate update statements against the same table to populate dimension keys. You have been asked to add one column. You have not read the procedure line by line yet.

What does the structure map tell you to check first?

- ☐ A Whether the three updates could be folded into the insert
- ☐ B Whether the truncate and insert are in a single transaction
- ☐ C Whether your new column needs to be in the insert or an update
- ☐ D Whether the updates are slowing the nightly run

◆ PAUSE AND DECIDE

What does the structure map tell you to check first?

- A

 Fold the updates in
A refactor you were not asked for, on a procedure you do not understand yet.
- B

 Transaction boundary
Truncate commits implicitly on most warehouses, so a mid-load failure leaves the table empty.
- C

 Where your column goes
You do need to answer this. It is a smaller question than whether the load is safe at all.
- D

 Are the updates slow
Probably yes, and it is not what the map just told you. Performance is a later module.

THE SIGNAL

The map surfaced a shape, and that shape has a known failure mode. Notice it before you add to it.

The transformation narration says a column is populated from a source field, upper cased, with nulls replaced by empty string. It describes this for every column in turn.

It reads well. It is roughly nine hundred words. You are no clearer on what the table is for.

What went wrong?

- ☐ A The procedure is genuinely just simple mapping
- ☐ B You got a translation of the syntax rather than an explanation
- ☐ C The prompt should have asked for business context
- ☐ D The target table was too large to scope in one prompt

◆ PAUSE AND DECIDE

What went wrong?

- A

 It really is simple
Possible, and you can test it in seconds. Ask for the row removal list and see if it is empty.
- B

 Translation, not explanation
Nine hundred words that could have been written without the data model. That is the tell.
- C

 Ask for business context
It would invent it. The code does not contain business context and neither does the model.
- D

 Scope was too large
Scope was right. The output was long because it padded, not because the table is complex.

THE SIGNAL

The row-removal half of that prompt is what carries the meaning. A missing filter list is the symptom.

You paste the procedure and ask what depends on the table. You get eight items back, including two views and a monthly finance report, all named plausibly.

You pasted no grep results and no consumer list. Your change ships Friday.

How much of that list can you rely on?

- ☐ A All of it. The names match our conventions closely
- ☐ B The views, since view names follow the table name pattern
- ☐ C None of it. Treat every item as a lead to verify
- ☐ D Only items it expressed high confidence about

◆ PAUSE AND DECIDE

How much of that list can you rely on?

- A

 All of it
The names match conventions because it generated them from your conventions. That is circular.
- B

 The views at least
The most seductive wrong answer. A plausible view name is still a generated view name.
- C

 None of it, verify each
Eight leads is genuinely useful. Eight facts would be dangerous. Know which you have.
- D

 The high confidence ones
Stated confidence is not calibrated against evidence it never had. It measures nothing here.

THE SIGNAL

The list is not worthless. It is a checklist to verify, and it took ten seconds to produce.

The structure map finds a column fed by different source fields in different union branches. You confirm a downstream bridge joins it with no branch filter.

The procedure has run nightly for four years. Nobody has ever reported a problem with the bridge.

What do you conclude?

- ☐ A It is fine. Four years without incident is strong evidence
- ☐ B It is a live bug. Fix the bridge before your change ships
- ☐ C It is a latent bug. Test for collisions, then decide
- ☐ D It is deliberate. Assume the original author knew something

◆ PAUSE AND DECIDE

What do you conclude?

- A

 Four years is evidence
Evidence that no collision has occurred yet. Not that the identifier spaces cannot overlap.
- B

 Live bug, fix it now
You do not know it is live. Fixing an unmeasured problem inside an unrelated change is its own risk.
- C

 Latent. Test, then decide
One count query settles it, and the answer changes what you do next entirely.
- D

 Assume it was deliberate
This is how every defect in this course survived. Absent evidence, deference is just a guess.

THE SIGNAL

One query tells you whether the identifier spaces overlap today. Run it before you argue about it.

Take this to work tomorrow

- ◆ Next unfamiliar procedure, run the structure map before you read a line of it.
- ◆ Narrate one target table. Check the row removal list came back populated.
- ◆ Before any blast radius question, spend ten minutes gathering grep and query history.
- ◆ Anything the code cannot explain goes in a comment, even if you never fix it.

THE PAYOFF

That last habit is what stops the next person inheriting the same mystery you did.

You can recover **what** the code does. You cannot recover **why**.

MODULE 05 · DOMAIN 3 - DEBUGGING AND PERFORMANCE

When The Numbers Are Wrong

Nothing failed. Nothing alerted. The total is thirty percent too high.

DOMAIN 3 of 5 MODULE 5 of 11 PROMPTS 3

Why this is not normal debugging

- ◆ **There is no stack trace.** Nothing threw. The query succeeded.
- ◆ **There is no failing test.** The output is well formed and in range.
- ◆ **You often cannot see the right answer.** Only that this one looks wrong.
- ◆ What you have instead is counts, and counts are enough.

THE DIFFERENCE

Debugging application code narrows by exception. Debugging data narrows by arithmetic.

Two ways to ask for help

THE DEFAULT

Describe the symptom

- Paste the query
- Say the total is too high
- Ask what is wrong with it
- Get four plausible theories

WHAT WE BUILD

Bisect with counts

- Count at every stage
- Find where the number changes
- Prove which join causes it
- One answer, with evidence

THE TIEBREAKER

Four theories feels like progress. It is four things to rule out, and it produced no evidence.

Do not ask what is wrong. Ask for the queries that will **tell** you.

What you actually type

You want queries back, not opinions.

The query below returns a total that is roughly 30% too high.
It runs without error. Do not tell me what you think is wrong.

Write a single diagnostic query that returns one row per stage
of this query's construction, with the row count at that stage.

Stages, in order:

1. driving table with its own filters only
2. after each join is added, one at a time, in query order
3. after the WHERE clause

Use UNION ALL with a label column so I can read it top down.

Then state which stage you expect the count to change at, and
why. I will run it and tell you the actual numbers.

THE LINES THAT MATTER

The last two lines force a prediction. A wrong prediction beats a vague theory.

The instrument, not the answer

Run it. Read down the labels.

```
01 SELECT '1 driving'      AS stage, COUNT(*) FROM wauf w
02   WHERE w.wotyp IN ('C1','C2','C3')
03 UNION ALL
04 SELECT '2 plus wopr',    COUNT(*) FROM wauf w
05   JOIN wopr o ON o.wonum = w.wonum
06   WHERE w.wotyp IN ('C1','C2','C3')
07 UNION ALL
08 SELECT '3 plus t401',    COUNT(*) FROM wauf w
09   JOIN wopr o ON o.wonum = w.wonum
10   JOIN t401 t ON t.wotyp = w.wotyp
11   WHERE w.wotyp IN ('C1','C2','C3')
```

THE RESULT

Predicted: no change at stage three. Actual: the count tripled. That is the whole investigation.

Reading the counts

- ◆ **Count unchanged.** That join is one to one. Move on.
- ◆ **Count increased.** Fan-out. The next prompt tells you which key is at fault.
- ◆ **Count decreased.** An inner join is dropping rows. Often the real bug.
- ◆ **Count matches the prediction.** Good. Keep going, and stay suspicious.

THE POINT

A decrease is the finding people miss. Nobody investigates a total that is too low.

You know which join. Now find out **why**.

Three things cause a fan-out

- ◆ **Genuinely one to many.** You joined a detail table for a filter and kept its rows.
- ◆ **Partial join key.** You joined on part but not on site. Half a key multiplies.
- ◆ **Source is not unique.** A text table with one row per language. A view with no dedup.
- ◆ The third one causes the most confusion, because the join looks correct.

THE POINT

Only the first is visible in the join clause. The other two need you to look at the data.

What you actually type

Name the join. Ask for proof, not explanation.

```
Row counts show the join to <table> triples the row count.  
The join condition is: <paste the ON clause>
```

Write three queries that will tell me which of these it is:

1. Is <table> unique on the join key? Group by the key, having count > 1, ordered by count descending, top 20.
2. Is the join key complete? Show me any column pair that appears in both tables but not in the ON clause.
3. Show me one example key with its duplicate rows in full.

Query three matters most. I want to see what differs between the duplicates, not be told what differs.

WHY QUERY THREE WINS

Seeing the duplicate rows side by side makes the cause obvious in seconds.

Two rows, one difference

Query three output for a single work order type.

```
01 wotyp    lang    wotypx
02 -----
03 C1       E       Corrective - Emergency
04 C1       F       Correctif - Urgence
05 C1       D       Korrektiv - Notfall
06
07 -- The ON clause joined on wotyp only.
08 -- The lookup holds one row per language.
09 -- Every work order was tripled. Hours tripled with it.
```

THE FINDING

The fix is four characters. Finding it without counts would have taken an afternoon.

Knowing when to stop asking and start again.

What happens across a long exchange

- ◆ Research on multi-turn SQL debugging finds quality degrades as the exchange goes on.
- ◆ Failed attempts stay in the context. They keep influencing what comes next.
- ◆ You start getting variations on earlier wrong answers, phrased more confidently.
- ◆ The fix is not a better follow-up. It is a clean start with what you learned.

THE EVIDENCE

This is measured behaviour, not folklore. Your fifteenth message is worse than your first.

How to recognise it happening

- ◆ The same suggestion returns, slightly reworded.
- ◆ Fixes start addressing symptoms you already ruled out.
- ◆ It apologises, then repeats the previous answer.
- ◆ Rough rule. Three failed attempts on one problem means restart.

THE HONEST PART

You will notice these signs and keep going anyway, because restarting feels like losing progress.

What you actually type

New session. This is the first message, not a follow-up.

I am debugging a query. Here is what is already established.

CONFIRMED BY QUERY:

- Join to t401 triples the row count
- t401 holds one row per language for each type code
- The ON clause does not filter language

RULED OUT BY QUERY:

- wopr join is one to one, count unchanged
- No duplicate work order numbers in the driving table

Given only that, write the corrected join clause and tell me what the row count should be after the fix.

THE HALF PEOPLE DROP

Both lists are findings. Carrying the second one is what stops the loop.

What you carry, what you leave

LEAVE BEHIND

The exchange

- Every failed attempt
- Every apology and revision
- Theories nobody tested
- The original query, if it is long

CARRY FORWARD

The findings

- Anything a query confirmed
- Anything a query ruled out
- The counts themselves
- The one question still open

THE TIEBREAKER

If a claim did not come from a query you ran, it does not go in the restart.

Counts cannot help you when the bug only
fires **some** nights.

Two clocks in one expression

Weekly snapshot. Runs nightly at 00:30 local.

```
01 v_snapshot_date INTEGER :=
02   CASE WHEN datepart('day', sysdate) = 1
03         THEN to_char(CONVERT_TIMEZONE('CST6CDT', SYSDATE-1), 'YYYYMMDD')
04         ELSE to_char(CONVERT_TIMEZONE('CST6CDT', SYSDATE), 'YYYYMMDD')
05   END;
06
07 -- The test uses SYSDATE. Raw. That is UTC.
08 -- The value uses the converted timestamp. That is local.
09 -- On the 1st, at 00:30 local, UTC already says the 1st.
10 -- Twelve times a year, the branch is chosen on the wrong day.
```

WHY NOBODY FOUND IT

Runs correctly on 353 nights. Picks the wrong branch on some of the other twelve.

Debugging something that is not there right now

- ◆ **Counts will not help.** Run them tonight and everything is correct.
- ◆ **Look for two sources of the same fact.** Two clocks, two calendars, two rounding rules.
- ◆ **Ask what input would make it differ.** That is the adversarial read, early.
- ◆ **Then check history.** If it fires monthly, the evidence is in the snapshot table.

THE POINT

The bug is not in the data you have. It is in the boundary between two ways of asking the same question.

The count triage shows the driving table at 890 thousand, unchanged after the first join, and 890 thousand after the second.

The final total is still thirty percent too high. Every join is one to one.

Where do you look next?

- ☐ A Re-run the triage. The counts must be wrong
- ☐ B The measure itself, not the joins
- ☐ C The source tables, for duplicate rows
- ☐ D The WHERE clause, for a filter that is too permissive

◆ PAUSE AND DECIDE

Where do you look next?

A Re-run the triage

The counts are simple and they agree with each other. Doubting your instrument first is rarely right.

C Duplicate source rows

That would have shown as a count increase. The triage already ruled it out.

B The measure itself

Row counts are correct, so the inflation is inside the calculation, not the row set.

D A permissive filter

A loose filter changes the count. The count is right, so this is not it.

THE SIGNAL

Correct row counts and a wrong total means the fault is in the arithmetic, not the join graph.

You have been debugging for forty minutes. The last three suggestions were variations on adding DISTINCT. You already tried that and it changed nothing.

You are fairly sure the problem is in the reservations join, but you have not proved it.

What do you do?

- ☐ A Tell it DISTINCT does not work and ask for something else
- ☐ B Restart with what you have confirmed and ruled out
- ☐ C Run the fan-out isolation queries on the reservations join first
- ☐ D Start a new session and paste the whole query again

◆ PAUSE AND DECIDE

What do you do?

- A

 Ask for something else
Three repeats is the signal to stop. A fourth request continues the loop you are already in.
- C

 Run the isolation queries
Turns the hunch into a confirmed finding, which is what makes a restart worth doing.
- B

 Restart with your findings
Right instinct, and premature. Your best lead is a hunch, so the restart would carry a guess.
- D

 New session, paste it all
A restart that carries no findings is just the same conversation with the evidence deleted.

THE SIGNAL

Restart when you have findings to carry. Get findings first, and the restart writes itself.

The triage shows the count dropping from 890 thousand to 604 thousand when a lookup table is joined. You were investigating a total that is too high.

The lookup is joined with an inner join to fetch a description.

What do you do with this?

- ☐ A Ignore it. You are looking for inflation, not loss
- ☐ B Note it and keep tracing the inflation
- ☐ C Stop and investigate the drop instead
- ☐ D Change it to a left join and re-run the triage

◆ PAUSE AND DECIDE

What do you do with this?

A Ignore it

You just found a third of the rows disappearing into a lookup join. That is not a side issue.

C Investigate the drop

An inner join to a description table should not remove rows. Something is missing from that lookup.

B Note it, keep going

Defensible and it defers the bigger finding. Nobody ever comes back to the noted item.

D Switch to left join

That restores the count and hides the cause. You still do not know why 286 thousand rows had no match.

THE SIGNAL

You were called about a number being too high. You just found a different number that is too low.

You find and fix the fan-out. The total drops to what the business expected. You are about to close the ticket.

The fix was a missing language filter in a join that has been in production for three years.

What else does this fix imply?

- ☐ A Nothing. The query is now correct
- ☐ B Every historical figure from this query was wrong too
- ☐ C The same lookup is probably joined elsewhere the same way
- ☐ D Both B and C, and both need saying out loud

◆ PAUSE AND DECIDE

What else does this fix imply?

- A

 Nothing further
The query is correct from tonight. Every number it produced before tonight was not.
- B

 History was wrong
True, and somebody made decisions on those numbers. It is half the answer.
- C

 It is joined elsewhere
Almost certainly, and one grep finds out. Also half the answer.
- D

 Both, said out loud
A three year old defect is never contained to the query where you happened to find it.

THE SIGNAL

The fix takes four characters. The conversation about three years of reported numbers takes longer.

Take this to work tomorrow

- ◆ Next wrong number, write the count triage query before you form a theory.
- ◆ When a count moves unexpectedly, ask for proof queries, not explanations.
- ◆ Keep a running list of confirmed and ruled out while you work.
- ◆ Three failed attempts on one problem. Restart. Set a timer if you have to.

THE PAYOFF

The confirmed and ruled out list costs nothing while you work and is the whole restart when you need it.

You never asked what was wrong. You asked for the **counts**.

MODULE 06 · DOMAIN 3 - DEBUGGING AND PERFORMANCE

Making It Faster

The query is correct. It takes eleven minutes. Both of those must still be true afterwards.

DOMAIN 3 of 5 MODULE 6 of 11 PROMPTS 3

A query that got four times faster and slightly **different**.

Why this goes wrong more than debugging does

- ◆ **Success is measurable.** The clock went down. That feels like proof.
- ◆ **Correctness is not checked.** Nobody reruns the old query to compare.
- ◆ **Rewrites change semantics quietly.** A join becomes a subquery. Nulls behave differently.
- ◆ **The old output disappears.** After one run you have nothing to compare against.

DO THIS FIRST

Save the current output before you change anything. That is the entire safety net.

Read the plan. Ranked by cost, not described.

What actually matters in a plan

- ◆ **Estimated rows against actual.** A large gap means the optimiser is working blind.
- ◆ **Data movement.** Broadcasts and redistributions on big tables dominate everything else.
- ◆ **Nested loops on large inputs.** Almost always a join condition the planner could not use.
- ◆ **The same table scanned repeatedly.** Cheap per scan, expensive in total.

THE POINT

Everything else in a plan is detail. These four account for most of the time in most slow queries.

What you actually type

Ranked, quantified, and no rewrites yet.

Below is the query and its execution plan. Do not rewrite anything yet.

Rank the plan steps by estimated share of total cost, highest first. Give me at most five. For each one state:

- Which line of the query it corresponds to
- Why it is expensive, in terms of the plan not the SQL
- Whether the row estimate looks wrong, and by how much

Then list separately any step whose cost comes from a stale or missing statistic rather than from the query.

If the plan does not support a claim, do not make it.

THE LINE THAT SHAPES IT

Capping it at five forces a ranking, not a walkthrough.

Four findings, one procedure

A consumption load with three time windows.

- ```
01 1. Temp table scanned four times
02 Three window CTEs plus a DISTINCT driver. Same rows each time.
03
04 2. Hash function called inside a join predicate
05 Computed per row. Planner cannot use it as a hash join key.
06
07 3. Date conversion applied per row, then filtered
08 TO_DATE on every row before the window predicate runs.
09
10 4. Unbounded aggregation over full history
11 No date predicate. Scans everything, every night.
```

## WHAT KIND OF FIX EACH IS

Only the first two are rewrites. The third is a predicate change and the fourth is a question for the business.

Rewrite it. But say what must not change.

# Pin these down before you ask

- ◆ **The grain.** One row per what. Same sentence from module three.
- ◆ **The row count.** State the number. It is the first thing you will check.
- ◆ **Null behaviour.** Which columns can be null, and what a null must produce.
- ◆ **Ordering guarantees.** If anything downstream depends on order, say so.

## THE POINT

An unconstrained rewrite is not wrong to ask for. It is just impossible to check.

# What you actually type

The constraints come before the request.

Rewrite the query below for performance. These must not change:

- Grain: one row per part per site
- Row count: 412,806
- Null handling: a part with no movements returns 0, not null

Target the four findings from the plan analysis above.

For each change you make, state in one line what it changes about execution and why the result set is unaffected.

If any change could alter the result, do not make it. List it separately as something I would have to decide about.

## WHY THE LAST BLOCK EXISTS

The last instruction is the important one. It gives it somewhere to put the risky ideas.

# Four scans become one

Three nested windows over the same rows.

```
01 -- Before: three CTEs, each scanning #wmov
02 cte_12: WHERE postdt BETWEEN <12 months> ... GROUP BY part,site
03 cte_24: WHERE postdt BETWEEN <24 months> ... GROUP BY part,site
04 cte_60: WHERE postdt BETWEEN <60 months> ... GROUP BY part,site
05
06 -- After: one scan, conditional aggregation
07 SELECT part, site,
08 SUM(CASE WHEN postdt >= <12 months> THEN net ELSE 0 END) AS m12,
09 SUM(CASE WHEN postdt >= <24 months> THEN net ELSE 0 END) AS m24,
10 SUM(net) AS m60
11 FROM #wmov WHERE postdt >= <60 months> GROUP BY part, site
```

## WHY IT IS SAFE

The windows are nested, so the widest one bounds the scan and the narrower ones are case expressions.

It argued that the results are the same.  
Now **prove** it.

# Why the argument is not enough

- ◆ The reasoning is usually correct. That is what makes this hard to insist on.
- ◆ It reasons about the query, not about your data.
- ◆ Edge cases live in the data. Nulls, duplicates, zero rows, boundary dates.
- ◆ One query settles it in the time it takes to read the argument.

## THE POINT

This is the same move as the debugging module. Ask for the check, not the claim.

# What you actually type

Both directions. That is the part people miss.

Below are the original query and your rewrite. Write me the queries that prove they return identical results.

1. Row count from each, side by side
2. Rows in original but not in rewrite
3. Rows in rewrite but not in original
4. Sum of each numeric measure from both, side by side

Queries two and three must both be included. One direction alone does not prove equivalence.

Then list the edge cases these four queries would NOT catch.

## THE HONEST LINE

The last line is the honest one. Four queries is a strong check, not a proof.

# Both directions, one result

If either count is non-zero, the rewrite is not equivalent.

```
01 SELECT 'in original only' AS side, COUNT(*) FROM (
02 SELECT * FROM original_result
03 EXCEPT
04 SELECT * FROM rewrite_result)
05 UNION ALL
06 SELECT 'in rewrite only', COUNT(*) FROM (
07 SELECT * FROM rewrite_result
08 EXCEPT
09 SELECT * FROM original_result)
10
11 -- Both zero, or the rewrite is not a rewrite.
```

## WHAT IT CAUGHT

It found 41 rows in the rewrite only. Parts with no movements now returned 0 instead of no row.

# Argued against proved

## WHAT YOU GET BY DEFAULT

### An argument

- Reasoning about the query
- Usually correct
- Free and instant
- Cannot see your data

## WHAT YOU SHOULD ASK FOR

### A test

- Reasoning about the result
- Correct or not, definitively
- Costs one query run
- Finds the forty one rows

## THE TIEBREAKER

The argument is worth reading. It is just not the thing you ship on.

The plan analysis ranks a broadcast join as the top cost. It also notes the row estimate on that step is 4,000 where the actual is 2.1 million.

You have a rewrite ready that removes the broadcast.

### What should you try first?

- ☐ A Apply the rewrite. The broadcast is the top cost
- ☐ B Update the table statistics and re-read the plan
- ☐ C Add a distribution key to the smaller table
- ☐ D Split the query into two steps with a temp table

### ◆ PAUSE AND DECIDE

## What should you try first?

- A

 Apply the rewrite  
*You would be optimising around a decision the planner made on bad information.*
- C

 Add a distribution key  
*Possibly right eventually. Still a change made while the planner has wrong numbers.*
- B

 Update stats, re-read  
*A 500x estimate error means the planner chose the broadcast blind. Fix the input first.*
- D

 Split into two steps  
*Materialising can help, and it is a workaround for a planner problem you have not diagnosed.*

### THE SIGNAL

A five hundred fold estimate error is the finding. The broadcast is the symptom of it.

The rewrite comes back with a clear justification for every change. It also lists one change separately, noting it could alter results for rows where a value is null.

That change accounts for most of the speed-up. Runtime goes from eleven minutes to ninety seconds.

### What do you do with the flagged change?

- ☐ A Take it. It flagged the risk, so the risk is known
- ☐ B Drop it and accept the smaller speed-up
- ☐ C Test whether any rows actually have that null
- ☐ D Take it and add a null guard to be safe

### ◆ PAUSE AND DECIDE

## What do you do with the flagged change?

- A

 Take it, risk is known  
*Knowing a risk exists is not knowing whether it applies. Those are different things.*
- B

 Drop it  
*Throws away most of the gain to avoid a risk you can measure in one count query.*
- C

 Test for the null  
*One count. If no rows have it, take the change. If some do, you have a real decision.*
- D

 Add a null guard  
*A guard changes behaviour too, and you still do not know what the old behaviour was.*

### THE SIGNAL

The prompt did its job by separating that change out. Your job is to settle it with a query.

The equivalence check returns zero rows in both directions. Counts match. Measure sums match to the penny.

A colleague asks whether the rewrite is proven correct.

### What is the accurate answer?

- ☐ A Yes. Both set differences are empty
- ☐ B Yes for today's data, which is not the same thing
- ☐ C No, because set difference ignores duplicate counts
- ☐ D No, until it has run in production for a week

### ◆ PAUSE AND DECIDE

## What is the accurate answer?

- A

 Yes, proven  
*Overstates it. You proved the two agree on one data set on one day.*
- C

 Duplicates are ignored  
*True of set difference, and the row counts matching already covers it here.*
- B

 Yes for today's data  
*The precise answer. Tomorrow may contain a boundary case today did not.*
- D

 Not until a week in production  
*Sets an unmeetable bar. By that logic nothing is ever verified before it ships.*

### THE SIGNAL

This is not pedantry. Date boundary logic is the classic case that passes today and fails next month.

The plan analysis says the largest single cost is a nightly full-table update on a history table, run to refresh dimension keys.

That update is not in the query you were asked to optimise. It is in the procedure that loads it.

### What is the right response?

- ☐ A Out of scope. Optimise the query you were given
- ☐ B Rewrite the update as part of the same change
- ☐ C Report it, with the cost figure, and ask how to proceed
- ☐ D Scope the update to only the current snapshot date

### ◆ PAUSE AND DECIDE

## What is the right response?

- A

 Stay in scope  
*You found the biggest cost in the workload and said nothing. Scope discipline taken too far.*
- B

 Rewrite it too  
*Expanding a performance ticket into a history table rewrite, unasked, is how changes go wrong.*
- C

 Report it with numbers  
*You have a measured finding outside your scope. Reporting it is the work; deciding is not yours.*
- D

 Scope it to today  
*Probably the right fix, and it is still a behaviour change to a history table made on your own.*

### THE SIGNAL

A finding outside your scope is still a finding. Handing it over with a number attached is enough.

# Take this to work tomorrow

- ◆ Before any optimisation, save the current output somewhere you can query it.
- ◆ Read the plan before you read the SQL. Ask for five steps, ranked.
- ◆ State grain, row count and null handling in the rewrite prompt itself.
- ◆ Run the equivalence check both directions. Every time, including when you are sure.

## THE PAYOFF

The first habit is the one that makes the last one possible.

Fast and correct. In that order of difficulty,  
and the **reverse** order of attention.

MODULE 07 · DOMAIN 3 - DEBUGGING AND PERFORMANCE

# Moving Between Engines

It compiled. It ran. It returned different rows.

DOMAIN 3 of 5    MODULE 7 of 11    PROMPTS 3

# Three kinds of migration problem

- ◆ **Does not compile.** Loud, immediate, and easy. You will fix these on day one.
- ◆ **Compiles, behaves differently.** Same function name, different rules.
- ◆ **Compiles, silently changes the result set.** Different rows. No error anywhere.
- ◆ Only the first kind gets caught by testing that the code runs.

## THE POINT

Every migration horror story you have heard is the third kind. Nobody loses sleep over a syntax error.

First, the easy class. Find everything that will not survive.

# What you actually type

Name both engines explicitly. Never say 'modern SQL'.

```
Source engine: <name and version>
```

```
Target engine: <name and version>
```

```
List every construct below that is not valid on the target.
```

```
For each one give:
```

- The line, quoted
- Why the target rejects it
- The target equivalent, if there is an exact one

```
If there is no exact equivalent, say so rather than offering
the closest thing.
```

```
Do not rewrite the code. I want the inventory first.
```

## THE LINE THAT MATTERS

Without the exact-equivalent rule, approximations arrive as translations.

# Four that will not compile

Legacy procedure, moving to a Postgres-family warehouse.

```
01 ISNULL(mstart, '') -- SQL Server. Use NVL or COALESCE
02 TRIM(a) + '|' + TRIM(b) -- + is arithmetic here. Use ||
03 DECODE(status, 'A', 1, 0) -- Oracle. Use CASE
04 WHERE ROWNUM <= 100 -- Oracle. Use LIMIT
05
06 -- All four fail loudly on the target. None is dangerous.
07 -- The second one is worth a second look, though.
08 -- If both operands are numeric, + is valid and silent.
```

## THE ONE TO NOTICE

Three of these throw. The fourth throws only when the operands happen to be text.

Now the dangerous class. Same syntax,  
different **meaning**.

# Where engines quietly disagree

- ◆ **Empty string and null.** Oracle treats an empty string as null. Most others do not.
- ◆ **Concatenation with null.** Oracle ignores it. Postgres family returns null.
- ◆ **Integer division.** Some engines return a decimal. Others truncate.
- ◆ **Implicit casts, date arithmetic, string truncation.** All differ, all silently.

## THE POINT

Every item here is valid syntax on both engines. Not one of them will raise anything.

# Concatenation, meeting a null

A hash key built from three components.

```
01 f_hash(source_key || UPPER(part_no) || UPPER(site_code))
02
03 -- On Oracle, if site_code is null:
04 -- null is treated as an empty string
05 -- the hash is built from the other two components
06
07 -- On a Postgres-family engine, if site_code is null:
08 -- the whole expression evaluates to null
09 -- the hash is null, and every join on it fails
10
11 -- Same line. Same syntax. Rows silently stop matching.
```

## WHAT ACTUALLY HAPPENS

The keys do not error. They come out null, the joins miss, and the rows land on a default value.

# What you actually type

Ask about what stays the same, not what changes.

```
Source engine: <name>. Target engine: <name>.
```

```
Find every expression below that is VALID on both engines but
may produce a different result. Ignore anything that fails to
compile on the target; I have those already.
```

```
For each one:
```

- What each engine returns
- The data condition needed to trigger the difference
- A query telling me whether that condition exists

```
Focus on nulls, empty strings, division and implicit casts.
```

## THE BULLET THAT SCALES IT

The third bullet turns theoretical risks into a short list of real ones.

# What it returned, and what survived testing

- ◆ **Eleven expressions flagged** as potentially behaving differently.
- ◆ **Seven ruled out by query.** The triggering condition does not occur in the data.
- ◆ **Three confirmed live.** Null site codes exist. Hash keys would break.
- ◆ **One undecidable.** Depends on data that has not arrived yet.

## THE RESULT

Eleven risks became three fixes and one thing to monitor. That is the point of asking for the queries.

Verification, when you cannot query both systems at once.

# Why the previous approach fails

- ◆ The two result sets live in different databases. You cannot subtract one from the other.
- ◆ Exporting both and diffing works, until the volumes are real.
- ◆ **Compare aggregates instead.** Counts, sums, and grouped checksums.
- ◆ **Then compare rows for a sampled slice.** Aggregates find drift. Rows find the cause.

## THE TWO LAYERS

Aggregates tell you that something differs. Only rows tell you what.

# What you actually type

Two queries. One per engine. Same shape.

```
I need to verify a migrated query returns the same results.
The two result sets are on different engines and cannot be
joined. Write me a matched pair of verification queries.
```

```
Each must return the same shape, so I can diff the outputs:
```

- A grouping key at a grain coarse enough to be readable
- Row count per group
- Sum of each numeric measure per group
- Count of nulls per nullable column, per group

```
Write them in the correct dialect for each engine. Do not
assume a function exists on both.
```

## THE ROW THAT EARNS ITS PLACE

The null counts are what catch the concatenation problem. Sums alone would not.

# Two kinds of verification

## SAME ENGINE

### Set difference

- Both directions, EXCEPT
- Definitive for the data tested
- One query
- Not available across engines

## ACROSS ENGINES

### Matched aggregates

- Two queries, diffed by eye
- Finds drift, not its cause
- Cheap at any volume
- Then sample rows where it differs

## THE TIEBREAKER

Neither is a proof. Both are far better than reading the translated code and agreeing with it.

The dialect audit finds a function used forty times across the codebase. The model offers a target equivalent and notes it is 'functionally similar'.

You have three weeks and two hundred procedures to migrate.

### What do you do with that phrase?

- ☐ A Accept it. Forty instances is too many to check individually
- ☐ B Ask specifically how it differs, before using it anywhere
- ☐ C Use it, and add it to a list to revisit if problems appear
- ☐ D Find a different equivalent that is exact

### ◆ PAUSE AND DECIDE

## What do you do with that phrase?

A Accept it

*Forty instances is the argument for checking, not against.  
One difference becomes forty defects.*

C Revisit if problems appear

*The problems this module is about do not appear. That is  
their defining feature.*

B Ask how it differs

*One prompt, applied once, covers all forty. This is the  
cheapest question in the module.*

D Find an exact one

*There may not be one. Assuming there is sends you looking  
for something that does not exist.*

### THE SIGNAL

Functionally similar is a phrase that means 'differs in ways I have not enumerated'.

The semantic drift prompt flags that null concatenation behaves differently. It gives you a query to check whether any hash key component is ever null.

You run it. Zero rows. No component is ever null in the current data.

### Is this risk closed?

- ☐ A Yes. The condition does not occur, so the behaviour cannot differ
- ☐ B No. Add a null guard to every hash expression anyway
- ☐ C No. The source system may start allowing nulls later
- ☐ D Yes for now, and it needs a constraint or a monitor

### ◆ PAUSE AND DECIDE

## Is this risk closed?

- A

 Closed  
*True today. It also means the safety of your hash keys now depends on an untracked assumption.*
- B

 Guard everything  
*Defensible, and it changes the hash values. Every key you have already stored would move.*
- C

 It might change later  
*Correct and unhelpful on its own. Every check in this module could be invalidated later.*
- D

 Closed, with a guard on it  
*Records the assumption in a place that will notice when it stops being true.*

### THE SIGNAL

You have discovered a load-bearing assumption. Write it into a constraint, not into your memory.

The matched aggregate queries agree on row counts and on every measure sum. The null counts differ on one column, by 1,400 rows.

That column is not used in any measure. It feeds a hash key.

### How serious is this?

- ☐ A Minor. The measures all agree, so the numbers are right
- ☐ B Serious. Those 1,400 rows will fail to join downstream
- ☐ C Unknown until you sample the rows and look
- ☐ D Cosmetic. Null counts often differ after a migration

### ◆ PAUSE AND DECIDE

## How serious is this?

A Minor

*The measures agree in this query. The hash feeds joins in other queries, where they will not.*

C Sample and look

*You should, to confirm. The mechanism is already identified, so this is verification not diagnosis.*

B Serious, joins will fail

*This is exactly the concatenation case, caught by the one check that sums would have missed.*

D Cosmetic

*Null counts differing is never cosmetic. It means two engines disagree about a value.*

### THE SIGNAL

This is the finding the null count row was added for. Sums and counts both passed it.

Migration is done. Everything verified. A colleague suggests running both systems in parallel for a month and comparing outputs daily.

It costs one engineer roughly two hours a week.

### What is the strongest case for doing it?

- ☐ A It catches anything the verification queries missed
- ☐ B It exercises data conditions that did not exist during testing
- ☐ C It gives the business confidence in the migration
- ☐ D It provides a rollback path if something goes wrong

### ◆ PAUSE AND DECIDE

## What is the strongest case for doing it?

- A

 Catches what was missed  
*True and vague. It does not say why a month of running would catch what a query did not.*
- B

 New data conditions  
*Names the actual mechanism. Every check you ran was against data that existed at the time.*
- C

 Business confidence  
*A real benefit and a weak argument. Confidence without a mechanism is just reassurance.*
- D

 Rollback path  
*Parallel running is not what gives you rollback. Keeping the old system running does.*

### THE SIGNAL

The undecidable item from the drift audit is exactly the thing a month of parallel running resolves.

# Take this to work tomorrow

- ◆ Name both engines and versions in every migration prompt. Never say 'standard SQL'.
- ◆ Run the drift audit separately from the compile audit. They find different things.
- ◆ For every drift finding, ask for the query that tells you whether it applies to your data.
- ◆ Include null counts per column in every verification. Sums will not catch a null key.

## THE PAYOFF

Seven of eleven risks were eliminated by running a query. That ratio is typical.

The dangerous code is the code that needed **no** translation.

MODULE 08 · DOMAIN 4 - TESTS, DOCS AND REVIEW

# Leaving It Better

Tests, monitors, and the documentation nobody has been able to afford until now.

DOMAIN 4 of 5    MODULE 8 of 11    PROMPTS 3

# What you know that nobody wrote down

By this point in a piece of work, you are holding a lot

- ◆ The grain, stated as a sentence, because you made it state one.
- ◆ Every place the document and the code disagree.
- ◆ Assumptions the query silently depends on. Nulls that must not appear.
- ◆ Which movement codes, which filters, which decisions nobody recorded.

## THE PROBLEM

All of that currently lives in your head and in a chat window you will close this afternoon.

The grain sentence is a test suite that has not been written yet.

# Four tests fall out of one sentence

One row per part per site

- ◆ **Uniqueness.** Part and site together must have no duplicates.
- ◆ **Referential.** Every key resolves. No rows parked on the default value.
- ◆ **Reconciliation.** The measure total ties back to the source.
- ◆ **Null and range.** Nullable columns behave, measures stay in plausible bounds.

## THE POINT

You did not need to invent any of these. They are implied by the grain and the join path.

# What you actually type

Give it the grain and the join path. It has everything it needs.

Grain: one row per part per site.  
Join path and target DDL are below.

Write the checks that would fail if this table were wrong:

1. Uniqueness on the grain key
2. Every dimension key resolves, none on the default
3. Measures reconcile to the source aggregate
4. Nullable columns behave as specified

Each check returns zero rows when healthy. No thresholds.

For each, name the defect it catches and one it would miss.

## THE CONVENTION THAT HELPS

Zero rows when healthy makes the whole suite readable at a glance.

# Zero rows means healthy

Four checks. Any output at all is a failure.

```
01 -- 1. Grain uniqueness
02 SELECT part_no, site, COUNT(*) FROM fct_usage
03 GROUP BY part_no, site HAVING COUNT(*) > 1;
04
05 -- 2. Unresolved dimension keys
06 SELECT COUNT(*) FROM fct_usage WHERE part_key = -2
07 HAVING COUNT(*) > 0;
08
09 -- 3. Reconciliation to source
10 SELECT ABS(f.total - s.total) FROM (...) f, (...) s
11 WHERE ABS(f.total - s.total) > 0.01;
```

## THE ONE THAT PAYS

Check two is the one that would have caught the null hash keys from the migration module.

A test and a monitor are not the same thing.

# Two different jobs

## A TEST

### Runs when code changes

- Known data, known answer
- Binary pass or fail
- Blocks a deployment
- Proves the code is right

## A MONITOR

### Runs when data arrives

- New data, unknown answer
- Threshold and trend
- Raises an alert
- Proves the assumption still holds

## THE TIEBREAKER

Everything in this course that broke silently would have been caught by a monitor, not a test.

# What you actually type

Feed it the assumptions you discovered, not the code.

Below are assumptions this pipeline depends on, discovered during development. None is enforced anywhere.

- site\_code is never null on the source table
- part numbers and contract ids never collide
- the movement type list has not changed

For each, write a monitor that runs after every load. Give me:

- The query, returning a count
- A threshold, and your reasoning for that number
- What a breach actually means, in business terms

If an assumption cannot be monitored, say so and explain why.

## THE INPUT THAT MATTERS

Feeding it assumptions rather than code is what makes this work. It cannot infer them.

# The assumption from module seven

Zero today. That is exactly why it needs watching.

```
01 -- Assumption: site_code is never null
02 -- Consequence if breached: hash key becomes null,
03 -- row fails every downstream join, lands on default key
04
05 SELECT COUNT(*) AS null_site_codes
06 FROM src_part_site
07 WHERE site_code IS NULL;
08
09 -- Threshold: 0. Not a tolerance. Any row breaks a key.
10 -- Current value: 0. Has been 0 for four years.
```

## WHY THIS ONE

An assumption that has held for four years is the one most likely to be undefended.

Now regenerate the document. From the code that actually runs.

# Why this is possible now and was not before

- ◆ **It was never a hard problem.** It was an expensive one.
- ◆ Documenting eight hundred lines by hand took a week nobody would fund.
- ◆ So it happened once, at go live, and never again.
- ◆ That is the entire mechanism behind every drift finding in this course.

## WHAT CHANGED

The economics changed. A regenerated mapping document is now an afternoon, not a week.

# What you actually type

Describe behaviour. Refuse to guess at purpose.

```
Generate a mapping document for the target table below, from
the deployed procedure. Column layout: <paste header>
```

```
For each target column give the source table, source column,
and the transformation exactly as the code performs it.
```

- Describe what the code does, never what it is for
- Quote literal values in full. Never summarise a code list

```
Then a section: BEHAVIOUR WITHOUT STATED REASON. Anything the
code does with no explanation available in the code.
```

```
Mark the document as generated from code, with today's date.
```

## THE RULE THAT MATTERS

Quote literals in full, or sixteen codes become 'a list' again.

# What the regenerated document is, and is not

- ◆ **It is accurate.** It describes the code that runs tonight, because it was read from it.
- ◆ **It is complete.** Every column, every literal, every filter.
- ◆ **It has no intent.** It cannot say why any rule exists.
- ◆ **It is not a specification.** Nobody agreed to it. It is a description.

## KEEP BOTH

Two artifacts, two jobs. The original says what was wanted. This one says what happens.

You generate the four grain tests. The uniqueness check fails immediately, returning 340 duplicate part and site pairs on a table that has been in production for three years.

The reconciliation check passes. Totals tie to source exactly.

### How do you read this?

- ☐ A The grain sentence is wrong. The real grain is finer
- ☐ B There is a fan-out and the totals are inflated
- ☐ C The uniqueness check is wrong for this table
- ☐ D The reconciliation check is too loose to detect it

◆ PAUSE AND DECIDE

## How do you read this?

- A

 The grain is finer  
*The likely answer. Duplicates plus correct totals means a real dimension you did not name.*
- B

 Fan-out, inflated totals  
*A fan-out would inflate the measures. Reconciliation passed, so the totals are not inflated.*
- C

 Bad check  
*The check tests exactly what the grain sentence claims. If it fails, the sentence is the suspect.*
- D

 Loose reconciliation  
*It reconciled to a penny. That is not a loose check by any reading.*

### THE SIGNAL

Duplicates with correct totals is the signature of a missing grain column, not a defect.

You propose a monitor with a threshold of zero on null site codes. A colleague objects that a hard zero will page somebody the first time a single bad row arrives.

They suggest a threshold of ten to reduce noise.

### What is the right response?

- ☐ A Agree. Ten is a reasonable noise filter
- ☐ B Hold at zero. One row breaks a key
- ☐ C Keep zero, but route it to a queue rather than a page
- ☐ D Set it to zero and review after a month

### ◆ PAUSE AND DECIDE

## What is the right response?

- A

 Threshold of ten  
*Ten rows with null keys is ten rows silently on a default. The threshold hides the thing being watched.*
- B

 Hold at zero  
*Correct on the number and it loses the argument. Their objection about paging is legitimate.*
- C

 Zero, but not a page  
*Keeps detection exact and fixes the real complaint, which was severity, not sensitivity.*
- D

 Review in a month  
*Defers a decision you can make now. The consequence of a null key is already known.*

### THE SIGNAL

Their objection was about severity. Answering it with sensitivity gives away the detection.

The regenerated mapping document is accurate and complete. Your manager suggests replacing the original document with it, since the original is two years out of date.

The original contains business context that the code does not.

### What do you recommend?

- ☐ A Replace it. Accurate beats out of date
- ☐ B Keep both, clearly labelled by what each one is
- ☐ C Merge them into a single corrected document
- ☐ D Keep the original and attach the generated one as an appendix

### ◆ PAUSE AND DECIDE

## What do you recommend?

- A

 Replace it  
*Destroys the only record of intent you have, to fix a problem about behaviour.*
- C

 Merge them  
*Produces one document where you can no longer tell which parts anyone agreed to.*
- B

 Keep both, labelled  
*They answer different questions. Labelling is what stops the generated one being read as agreed.*
- D

 Generated as appendix  
*Close, and it subordinates the accurate document to the stale one. Wrong way round.*

### THE SIGNAL

The moment you cannot tell description from specification, you have lost both.

The BEHAVIOUR WITHOUT STATED REASON section of the generated document lists nine items. Seven are trivial. Two are substantive filters excluding whole categories of data.

You are at the end of the sprint.

### What do you do with the two substantive items?

- ☐ A Leave them in the document. It is recorded, which is the point
- ☐ B Raise them as tickets to investigate later
- ☐ C Ask the two longest-serving people on the team, today
- ☐ D Remove the filters, since nobody can justify them

### ◆ PAUSE AND DECIDE

## What do you do with the two substantive items?

A Recorded is enough

*Better than nothing and it preserves the mystery rather than resolving it.*

C Ask people today

*Institutional memory walks out of the building. It is free right now and gone in a year.*

B Tickets for later

*The honest version of A. Nobody has ever picked up the ticket that says find out why this filter exists.*

D Remove the filters

*Unexplained is not unjustified. That distinction is the whole course.*

### THE SIGNAL

The document made the questions askable. Asking them is a ten minute conversation with a deadline.

# Take this to work tomorrow

- ◆ Turn your grain sentence into four checks. It takes one prompt.
- ◆ Write down every assumption you found, then generate a monitor for each.
- ◆ Regenerate the mapping document for one table. Mark it as generated, with the date.
- ◆ Whatever comes back unexplained, ask a human today rather than filing it.

## THE PAYOFF

None of this is a project. It is one afternoon at the end of work you have already done.

Drift is not getting solved. It is getting  
**cheap** to correct.

# Reviewing What It Gave You

Every module so far assumed you could tell when the output was wrong.

# What the research actually found

- ◆ **A randomised trial.** Experienced developers were 19% slower using AI tools.
- ◆ **They believed the opposite.** Predicted 24% faster. Reported about 20% faster afterwards.
- ◆ **Defect rates.** AI-coauthored changes carry noticeably more issues than others.
- ◆ **And yet.** Most developers do not merge without reviewing. The instinct is right.

## THE POINT

The gap between being slower and feeling faster is the finding. It applies to correctness too.

Fluent output reads as **correct** output. It always has.

# Why 'review this code' does nothing

- ◆ **It has no target.** A general request gets a general answer.
- ◆ **It defaults to style.** Naming, formatting, comments. All true, none important.
- ◆ **It has already agreed with itself.** It wrote this. It is not a fresh reader.
- ◆ **Name the defect class** and the hit rate changes completely.

## THE POINT

Asking it to check its own work is asking the same reasoning to reach a different conclusion.

Name the defect. Do not ask for a review.

# What you actually type

One class per pass. Run it several times.

Check the code below for ONE class of defect only.  
Ignore style, naming, formatting and performance.

**This pass: operator precedence in WHERE clauses.**

For every WHERE clause mixing AND and OR, show me how it actually parses, and whether that matches the apparent intent based on the indentation.

Quote each clause. State the parse. State the likely intent.  
If they match, say 'matches' and move on.

**Do not report anything outside this defect class.**

## HOW TO USE IT

Run this five times with five different classes. It beats one general review every time.

# Defect classes worth naming

One pass each. Pick the ones your codebase actually suffers from

- ◆ **Operator precedence** in clauses mixing AND and OR.
- ◆ **Null propagation** through concatenation, arithmetic and comparison.
- ◆ **Join cardinality** against the declared grain.
- ◆ **Literal consistency** for the same code list across procedures.
- ◆ **Dialect drift** where a function is not native to the target engine.

## THE RESULT

Every one of these appeared somewhere in this course. None would be found by a general review.

# Null propagation, found by naming it

A general review pass read straight past this.

```
01 -- The dimension builds the key like this:
02 f_hash(src_key || UPPER(part) || NVL(UPPER(val_type),''))
03
04 -- The fact builds the same key like this:
05 f_hash(src_key || UPPER(part) || NVL(UPPER(val_type)),'')
06
07 -- Look at the closing bracket. The empty string is outside
08 -- the NVL, so NVL gets one argument and never defaults.
09 -- Null valuation type gives a null key. The join misses.
```

## WHY TARGETING WORKS

Two characters apart, in two files, three hundred lines from each other.

Stop asking whether it is right. Ask it to show you it is **wrong**.

# Same code, two questions

## WHAT YOU NORMALLY ASK

### Is this correct?

- Invites agreement
- Finds reasons it works
- Caveats arrive as reassurance
- Confirms what you already hoped

## WHAT TO ASK INSTEAD

### Show me it breaks

- Invites a counter-example
- Finds the input that fails
- Caveats arrive as conditions
- Gives you something to test

## THE TIEBREAKER

The second question returns data conditions. Data conditions are checkable. Opinions are not.

# What you actually type

Assume it is broken. Make it find how.

Assume the query below is wrong. Your job is to find the input that breaks it. Do not defend it.

Give me five data conditions that would make this return an incorrect result, ordered by how likely they are in a real warehouse. For each:

- The condition, stated concretely
- What the query would return
- What it should return
- A query that tells me whether the condition exists

If you cannot find five, say so. Do not pad the list.

## THE BULLET THAT MATTERS

The fourth bullet turns the whole exercise from speculation into a morning's checking.

Last one. Make it tell you what it decided  
**for** you.

# What you actually type

Run this at the end of every session that produced code.

```
You have just written this query. Before I use it, list every
decision you made that I did not specify.
```

- Anything ambiguous in the spec that you resolved
- Any behaviour you chose where more than one was valid
- Anything you assumed about the data

```
For each, state what you chose and what the alternative was.
```

```
Then: which of these decisions would change the output if I
picked the other option? Order by size of effect.
```

```
If you made no such decisions, say so plainly.
```

## WHAT TO DO WITH IT

This is the comment block that goes at the top of the query. It costs twenty seconds.

# What review cannot catch, at all

- ◆ **A rule that is missing entirely.** Nothing in the code points at what is absent.
- ◆ **A code list that is stale.** The syntax is fine. The world moved.
- ◆ **The wrong business definition.** Correctly implemented, and not what was wanted.
- ◆ **All three need a person,** or a query against real data. Never another review pass.

## THE BOUNDARY

If a second review pass would have found it, the first pass was badly targeted. If not, stop reviewing.

You ask for a general review of a 200 line procedure. You get eleven observations. Nine are about naming and formatting. Two suggest adding error handling.

The procedure contains a WHERE clause with a precedence bug you already know about.

### What does this tell you?

- ☐ A The model is not capable of finding precedence bugs
- ☐ B The prompt had no target, so it defaulted to surface issues
- ☐ C The procedure is too long for one review pass
- ☐ D Precedence bugs are genuinely hard to detect

### ◆ PAUSE AND DECIDE

## What does this tell you?

- A

 Not capable  
*Test it. Ask the targeted precedence prompt on the same code and watch it find the bug.*
- B

 No target, surface answer  
*A general request gets a general answer, and style is the easiest general answer available.*
- C

 Too long  
*Length matters and it is not the issue here. A targeted pass finds it at the same length.*
- D

 Genuinely hard  
*It is hard for a human reading quickly. It is easy when you name it as the thing to look for.*

### THE SIGNAL

The same model, the same code, one changed instruction. That is the finding worth keeping.

The adversarial read returns five data conditions that would break the query. You run the check queries. Four conditions do not exist in your data. The fifth does.

The fifth is ranked as the least likely of the five.

### What do you conclude about the ranking?

- ☐ A The ranking was wrong and should be ignored in future
- ☐ B The ranking was a prior, and your data is the evidence
- ☐ C The check query for the fifth condition is probably faulty
- ☐ D Four out of five is a good hit rate

#### ◆ PAUSE AND DECIDE

## What do you conclude about the ranking?

- A

 Ignore rankings  
*Overcorrects. The ranking told you what order to check in, which is all it was ever for.*
- B

 A prior, updated by data  
*It ranked by likelihood in a typical warehouse. Yours is not typical, and now you know how.*
- C

 Faulty check query  
*Worth a glance and it is not the interesting reading. Doubting the result you did not expect is a habit.*
- D

 Good hit rate  
*Misreads the exercise. Ruling four out is the value, not a score to be graded on.*

### THE SIGNAL

The ranking sets your checking order. Your data decides. That is the division of labour.

The assumption disclosure returns four decisions. Three are trivial. The fourth says it assumed a code list was current, because the spec did not date it.

You recognise this. It is the same code list from the reconciliation module.

### What is the right response?

- ☐ A Nothing new. You already knew about that list
- ☐ B Put all four in a comment at the top of the query
- ☐ C Go back and get the list verified before shipping
- ☐ D Both B and C

### ◆ PAUSE AND DECIDE

## What is the right response?

- A

 Nothing new  
*True for you and false for everyone who reads this query after you leave.*
- B

 Comment all four  
*Cheap and permanent. Even the trivial three cost nothing to carry.*
- C

 Verify before shipping  
*Right, and it is the second time this list has surfaced. That is a pattern, not a coincidence.*
- D

 Both  
*Record it and resolve it. Neither one substitutes for the other.*

### THE SIGNAL

The same unverified list has now been flagged by two different prompts in two different modules.

You have run targeted reviews for five defect classes, an adversarial read, and assumption disclosure. Everything comes back clean.

The business owner says the number still looks wrong to her. She cannot say why.

### What do you do?

- ☐ A Show her the review results. The query is verified
- ☐ B Run more review passes with different defect classes
- ☐ C Ask her which specific figure looks wrong, and reconcile that one by hand
- ☐ D Rebuild the query from scratch and compare

### ◆ PAUSE AND DECIDE

## What do you do?

A Show the review results

*You verified the code does what it says. She is telling you it says the wrong thing.*

C Reconcile one figure by hand

*Her instinct is evidence from outside the code. One traced figure turns it into a finding.*

B More review passes

*Review cannot find a missing rule or a wrong definition. More passes will keep coming back clean.*

D Rebuild from scratch

*Expensive, and it would reproduce the same misunderstanding from the same documents.*

### THE SIGNAL

She has information the code does not contain. That is the only kind of evidence review cannot reach.

# The SQL arc, in one line each

| MODULE                     | THE ONE THING                                                   |
|----------------------------|-----------------------------------------------------------------|
| Context and reconciliation | Documents drift. Diff them against code before you trust either |
| Spec to SQL                | Declare the grain before you write anything                     |
| Inherited code             | You can recover what it does. Never why                         |
| Wrong numbers              | Ask for counts, not diagnoses                                   |
| Performance                | Argued equivalent is not proved equivalent                      |
| Migration                  | The dangerous code is the code that needed no translation       |

## WHAT IS LEFT

Two modules left. Everything so far assumed the code was SQL.

# Take this to work tomorrow

- ◆ Replace 'review this' with one named defect class. Run it five times.
- ◆ Ask for the input that breaks it, not whether it is correct.
- ◆ Run assumption disclosure at the end of every session that produced code.
- ◆ When someone says the number looks wrong, trace one figure by hand. Believe them first.

## THE PAYOFF

The last one is not a prompting technique. It is the most valuable habit in the course.

None of these prompts decided anything.  
**You** did.

MODULE 10 · DOMAIN 5 - BEYOND SQL

# When It Is Not SQL

The same job, written in Spark. Three failure modes that have no SQL equivalent.

DOMAIN 5 of 5    MODULE 10 of 11    PROMPTS 3

# What changes, and what does not

## STILL TRUE

### Carries over

- Grain decides everything
- Counts bisect the problem
- Documents drift from code
- Argued is not proved

## NEW HERE

### No SQL equivalent

- Types coerce silently on join
- Errors surface far from the cause
- Code depends on state not in the file
- Config and code disagree

## THE TIEBREAKER

The engine will not check your join keys. In SQL it at least tries.

Assert the schema. The engine will not do it for you.

# Two types, no error

Parts read from Parquet. Movements read from a CSV export.

```
01 parts = spark.read.parquet("/lake/pmats") # partno: bigint
02 moves = spark.read.csv("/drop/wmov", header=True) # partno: string
03
04 df = parts.join(moves, on="partno", how="left")
05
06 -- Spark casts one side to make the join work.
07 -- No error. No warning. The job succeeds.
08 -- Leading zeros in the CSV never match the bigint.
09 -- Result: consumption is zero for 40% of parts.
```

## WHY IT IS WORSE HERE

In SQL this often throws. In Spark it is a successful run with a wrong answer.

# What you actually type

Ask for assertions, not for a review of the join.

```
Below is a PySpark job. Before any transformation runs, I want
the schema asserted rather than assumed.
```

```
For every DataFrame created by a read, write an assertion that
fails loudly if the schema is not what the code expects.
```

- Column present, and its exact dtype
- For every join key: same dtype on both sides
- Row count is greater than zero

```
Then list every join where Spark would silently cast one side,
and say which side it would cast.
```

## WHY ASSERT RATHER THAN REVIEW

Assertions are permanent. A review finds it once, then the file changes.

The error is not where the stack trace points.

# Why the trace misleads

- ◆ **Transformations do not run.** They build a plan. Nothing executes yet.
- ◆ **The action runs everything.** A count or a write, hundreds of lines later.
- ◆ **The trace names the action.** Not the transformation that was wrong.
- ◆ **So paste the whole job.** A model given only the failing line cannot find the cause.

## THE POINT

This is why 'here is my error, here is the line' works in SQL and fails here.

# What you actually type

Give it the whole lineage. Ask it to walk backwards.

This PySpark job fails at the write on line 214. The full job is below. Do not assume the fault is at line 214.

Walk backwards through the lineage from the action. List every transformation contributing to it, in reverse order.

For each, state:

- What it assumes about its input
- Whether that assumption is checked anywhere
- What the error would look like if it were the cause

Then rank them by how well each explains the actual error.

## THE BULLET THAT NARROWS IT

Most candidates would produce a different error. That eliminates them fast.

# The Spark version of count triage

- ◆ **Insert an action between stages.** A count forces execution up to that point.
- ◆ **Count after every read, and after every join.** Same bisect as module five.
- ◆ **It costs a full recompute each time.** Cache first, or sample the input.
- ◆ **Then the counts read exactly like SQL.** Up means fan-out, down means a filter.

## IN PRACTICE

Same technique, one extra step. You have to force execution before you can count anything.

The notebook is not the program.

# It runs for you. Only for you.

A notebook that has been open for three weeks.

```
01 # Cell 4
02 df = df.filter(F.col("site").isin(active_sites))
03
04 # active_sites was defined in a cell that has been deleted.
05 # It still exists in the kernel. The notebook still runs.
06 # On a fresh kernel it fails. In the scheduler it fails.
07
08 # Cell 9
09 spark.conf.set("spark.sql.shuffle.partitions", "8")
10 # Set after the job was tuned. Affects every cell above it
11 # on the next run, and none of them on this one.
```

## THE GAP

A model reading this file sees a program. You are running something else entirely.

# What you actually type

Ask what a fresh kernel would do.

Below is a notebook, exported in cell order. Assume it runs top to bottom on a fresh kernel, by a scheduler, with no prior state.

List everything referenced before it is defined, in that order.  
Include variables, imports, temp views and config settings.

Then list anything defined after it is first used, and which earlier cells would behave differently on a rerun.

Then: which cells depend on tables this notebook does not create.

## WHY IT WORKS

Cell order is a fact in the file. Execution order is not.

Consumption comes back as zero for about 40% of parts. The job succeeded. No errors in the log.

The same logic in SQL, against the same source data, returns correct figures.

### What do you check first?

- ☐ A The filter logic, since a filter is removing those parts
- ☐ B The dtypes on both sides of every join key
- ☐ C Whether the CSV source is missing those rows
- ☐ D Row counts after each stage

### ◆ PAUSE AND DECIDE

## What do you check first?

**A** The filter logic

*A filter would remove rows. These parts are present with a zero, which is a failed match.*

**C** Missing source rows

*The SQL version reads the same source and finds them. So they are there.*

**B** Join key dtypes

*SQL works and Spark does not, on identical data. That points at the layer SQL types for you.*

**D** Stage counts

*Useful, and a left join keeps the rows. Counts would look correct throughout.*

### THE SIGNAL

Zero rather than absent is the tell. The row survived the join and matched nothing.

A job fails at a write on line 214 with a null pointer error. You paste lines 205 to 214 and the error message.

You get three suggestions. None of them fixes it.

### What went wrong with your approach?

- ☐ A The error message was too generic to act on
- ☐ B You pasted the action, not the transformations that built it
- ☐ C You should have pasted the full stack trace
- ☐ D The model cannot debug Spark reliably

### ◆ PAUSE AND DECIDE

## What went wrong with your approach?

**A** Generic error

*It is generic, and a generic error with full lineage is still diagnosable.*

**C** Full stack trace

*The trace describes Spark internals below your action. It still will not name your transformation.*

**B** Pasted the action only

*Ten lines around the write contain none of the transformations that produced the bad column.*

**D** Cannot debug Spark

*Same conclusion people reach about SQL review. It is the input that was wrong, not the tool.*

### THE SIGNAL

In SQL, the failing line is usually the wrong line. In Spark, it is almost never the wrong line's fault.

A notebook has run daily for two months. You schedule it and it fails immediately on a name error.

It still runs perfectly when you execute it yourself.

### What is the most likely cause?

- ☐ A The scheduler has a different Python environment
- ☐ B Something is defined in your kernel and not in the file
- ☐ C Cell execution order differs under the scheduler
- ☐ D A permissions difference on the service account

### ◆ PAUSE AND DECIDE

## What is the most likely cause?

- A

 Different environment  
*That usually raises an import error, not a name error on your own variable.*
- B

 Kernel-only definition  
*A name error plus works-for-me is the signature of state that is not in the file.*
- C

 Different cell order  
*Schedulers run top to bottom. It is your own order that has been non-linear for two months.*
- D

 Permissions  
*Permissions fail with access errors. They do not produce a name error.*

### THE SIGNAL

Two months of daily runs is how long the kernel has been alive, not how long the code has worked.

You add schema assertions to a job. On the first scheduled run, an assertion fails: a column that was a string is now a decimal.

The job would have run fine. Spark would have handled the cast.

### Was the assertion worth having?

- ☐ A No. It failed a job that would have worked
- ☐ B Yes. The upstream schema changed and nobody told you
- ☐ C Only if the cast would have changed a result
- ☐ D Yes, but the assertion should warn rather than fail

### ◆ PAUSE AND DECIDE

## Was the assertion worth having?

- A

 Not worth it  
*Judges the assertion by this one cast. Its job is telling you the contract moved.*
- B

 Worth it, schema changed  
*An upstream change reached you without notice. Next time it may not be a harmless cast.*
- C

 Only if results changed  
*You cannot know that in advance, which is exactly why the assertion runs first.*
- D

 Warn, do not fail  
*A warning in a scheduled log is a warning nobody reads. This is the monitor argument again.*

### THE SIGNAL

Same shape as the zero threshold from module eight. Detection stays exact; routing is a separate question.

# Take this to work tomorrow

- ◆ Add dtype assertions on both sides of every join key. Today, in one job.
- ◆ When something fails at an action, paste the whole lineage. Never the failing line.
- ◆ Run every notebook once on a fresh kernel before you schedule it.
- ◆ Counts still bisect. You just have to force execution first.

## THE PAYOFF

The first one takes ten minutes and removes the most common silent failure in Spark.

The job is correct. Now something has to  
**run** it.

MODULE 11 · DOMAIN 5 - BEYOND SQL

# Something Has To Run It

Orchestration failures are correctness failures wearing a scheduling costume.

DOMAIN 5 of 5    MODULE 11 of 11    PROMPTS 3

A pipeline that fails is fine. A pipeline that succeeds on **stale** input is not.

# Three ways orchestration produces wrong data

- ◆ **Order.** A task runs before its input is ready and reads what was there yesterday.
- ◆ **Retries.** A task runs twice and appends twice. Nothing errors.
- ◆ **Backfill.** A rerun for last month uses today's lookup values.
- ◆ **All three succeed.** Green in the interface, wrong in the warehouse.

## THE POINT

Every one of these looks identical to a healthy run from the scheduler's point of view.

The dependency graph is not the data  
dependency graph.

# The edge nobody drew

Three tasks. Two declared dependencies.

```
01 load_parts >> load_usage >> build_report
02
03 # But inside build_report:
04 spark.table("fs_dw.dim_site_flat")
05
06 # dim_site_flat is loaded by a DAG in another repository,
07 # on its own schedule, roughly 40 minutes later.
08 # There is no dependency between them at all.
09
10 # Most days it wins the race. Some days it does not.
```

## THE RULE

Reading a table is a dependency whether or not anybody declared it.

# What you actually type

Two graphs. Ask for the difference.

Below is a DAG definition and the code for every task in it.

Build two lists and compare them.

1. Dependencies DECLARED in the DAG
2. Every table, path or topic each task READS

Then: any object read by a task where nothing upstream in this DAG writes it. Those are undeclared dependencies.

For each, say what the task would read if it ran early.

Do not assume an external system is safe because it is external.

WHERE YOU HAVE SEEN THIS

This is the reconciliation prompt again. Two descriptions of the same thing, diffed.

Ask what happens when it runs **twice**.

# Why retries are dangerous by default

- ◆ **Retries are on.** Usually two or three, configured once and forgotten.
- ◆ **Failure is rarely clean.** A task can write half its output, then die.
- ◆ **The retry appends.** Now you have one and a half loads.
- ◆ **Truncate and reload is accidentally safe.** Append and merge are not.

## THE CONNECTION

You saw this in module four. Truncate then insert is unsafe on failure and safe on retry.

# What you actually type

Ask about the second run, not the first.

Below is a task that writes to a warehouse. Assume it fails partway through the write and is retried automatically.

Walk through what the warehouse contains after the retry.

Assume the first attempt committed some of its output.

Then answer:

- Is the result identical to a single clean run?
- If not, what is duplicated and how would I detect it?
- What is the smallest change that makes the retry safe?

Also: does this task depend on the wall clock?

THE BLOCK THAT SURPRISES PEOPLE

A job filtering on 'today', retried after midnight, reads a different day.

Last prompt. What does a backfill actually reproduce?

# What you think a backfill does

## THE EXPECTATION

### Recreates the past

- Same input as that day
- Same lookups as that day
- Same code as that day
- Same output as that day

## WHAT RUNS

### Today, with an old date

- Current state of the source
- Current lookup values
- Current code
- A number nobody can reproduce

## THE TIEBREAKER

Only the date parameter goes back in time. Everything else stays where it is.

# What you actually type

The last prompt in the course, and the least run.

```
Below is a task and its DAG. I need to backfill it for a date
three months ago.
```

```
List everything this task reads that is NOT bounded by the
execution date. Those are the things that will use today's
values rather than the values from that date.
```

```
For each, tell me:
```

- Whether a historical version exists at all
- What would differ in the output if it has changed

```
Then: is the output of this backfill comparable to the original
run, or is it a different number that looks the same?
```

## THE QUESTION THAT MATTERS

That closing question is the one to actually answer before you press the button.

A report is wrong roughly one morning in ten. On the bad mornings, one site is missing. Rerunning the report task fixes it every time.

The DAG shows the report depending on the two loads that feed it.

### What is the most likely cause?

- ☐ A An intermittent failure in one of the two upstream loads
- ☐ B An undeclared dependency on something outside this DAG
- ☐ C A retry duplicating rows and confusing the aggregate
- ☐ D A timezone boundary in the report's date filter

### ◆ PAUSE AND DECIDE

## What is the most likely cause?

- A

 Upstream failure  
*A failed upstream task would block the report, not let it run with partial data.*
- B

 Undeclared dependency  
*Rerunning later fixes it. That is a race, and a race means an edge nobody drew.*
- C

 Retry duplication  
*Duplication adds rows. This is a missing site, which is the opposite symptom.*
- D

 Timezone boundary  
*Would cluster around month or day boundaries. One morning in ten is not that pattern.*

### THE SIGNAL

Fixed by rerunning, with nothing else changed, means the input was simply not ready the first time.

The dependency prompt finds four undeclared reads. Three are reference tables that change once a year. One is a dimension loaded daily by another team.

Adding cross-DAG dependencies is a two week piece of work.

### What do you do first?

- ☐ A Declare all four properly. Do it once and do it right
- ☐ B Declare the daily one. Document the other three
- ☐ C Add a sensor that waits for the daily dimension
- ☐ D Move the report later in the day to avoid the race

### ◆ PAUSE AND DECIDE

## What do you do first?

- A

 Declare all four  
*Two weeks of work, and three of the four carry almost no risk. Poor use of the finding.*
- B

 Daily one, document rest  
*Right prioritisation and it leaves the actual race in place while you build.*
- C

 Sensor on the dimension  
*Closes today's race in an afternoon. Then do B properly with the pressure off.*
- D

 Move it later  
*Buys time and does not remove the race. The other team will get slower eventually.*

### THE SIGNAL

Frequency of change is your risk ranking. Annual reference data is not what is breaking your report.

Finance asks you to backfill six months of a report because a bug was fixed. The backfill runs clean and the numbers differ from the originals by small amounts.

Nobody can say which set is correct.

### What is the underlying problem?

- ☐ A The bug fix changed the numbers, which is expected
- ☐ B The dimensions have changed, so neither run is reproducible
- ☐ C The backfill ran with the wrong execution dates
- ☐ D The original runs were wrong and the backfill is right

### ◆ PAUSE AND DECIDE

## What is the underlying problem?

- A

 The fix changed them  
*Partly true and it does not explain why nobody can tell which is right.*
- B

 Dimensions changed  
*Two variables moved at once. You cannot attribute the difference to either.*
- C

 Wrong dates  
*Would produce large obvious differences, not small ones spread evenly.*
- D

 Backfill is right  
*It uses today's dimensions for a period six months ago. That is not obviously right.*

### THE SIGNAL

You changed the code and the reference data at the same time. Neither number is now defensible.

You have applied everything in this course to a pipeline. Reconciled, tested, monitored, dependencies declared, backfill behaviour documented.

A new engineer joins and asks where to start reading.

## What do you hand them?

- ☐ A The original mapping document
- ☐ B The regenerated mapping document
- ☐ C The monitors, since they encode the live assumptions
- ☐ D The list of things nobody could explain

### ◆ PAUSE AND DECIDE

## What do you hand them?

A Original document

*It says what was wanted in 2023. Useful later, misleading as a first read.*

C The monitors

*Genuinely good and narrow. It tells them what is fragile, not what the system is.*

B Regenerated document

*Accurate about behaviour and it reads as settled. It hides every open question.*

D The unexplained list

*The only artifact that shows where knowledge actually ends. Everything else looks complete.*

### THE SIGNAL

Every other document reads as finished. Only that list tells the truth about what nobody knows.

# Beyond SQL, in one line each

| WHERE           | THE ONE THING                                                |
|-----------------|--------------------------------------------------------------|
| Spark types     | The engine will not check your join keys. Assert them        |
| Lazy evaluation | The error is never where the trace points. Paste the lineage |
| Notebooks       | Cell order is in the file. Execution order is not            |
| Dependencies    | Reading a table is a dependency whether declared or not      |
| Retries         | Ask what happens on the second run, not the first            |
| Backfill        | Only the date goes back in time. Everything else stays       |

## THE PATTERN

Different mechanisms, same shape. Something succeeded and the data is wrong.

# Take this to work tomorrow

- ◆ Run the dependency diff on one pipeline. You will find at least one undeclared read.
- ◆ Pick your most important task and ask what a retry does to it.
- ◆ Before the next backfill, list what is not bounded by the execution date.
- ◆ Keep the list of things nobody can explain. It is the most useful document you own.

## THE PAYOFF

The first one takes twenty minutes and finds something in almost every pipeline.

Thirty two prompts. Not one of them decided anything.